

系统之间的接口，它并没有规定引导加载程序该如何做，只是规定与正在装入的操作系统该如何交互。针对当前 x86 体系结构和免费的 32 位操作系统，多重引导规范为引导加载程序提供了一种将配置信息传送到操作系统的标准方式。操作系统的映像可以是任何类型的（ELF 或其他特别的类型），但在其前 8 K 中必须包含多引导头和魔数 0x1BADB002。由于某些操作系统并不将操作所需的所有程序都装入可引导的内核映像，因此多重引导兼容的装载程序还应该提供一种方法，方便操作系统在引导时使用辅助的引导模块和驱动程序，这样做通常可以将引导内核模块化，并将其大小控制在便于管理的规范内。

多重引导规范规定，当引导加载程序激活操作系统时，系统必须运行于特定的 32 位实模式状态，这样操作系统才能在需要时成功地回调 BIOS。最后，引导加载程序必须为操作系统提供填满基本机器数据的数据结构。接下来看看多重引导信息的数据结构：

```
-----
typedef struct multiboot_info
{
    ulong flags;    // indicate following fields
    ulong mem_lower; // if flags[0], amnt of mem < 1M
    ulong mem_upper; // if flags[0], amnt of mem > 1M
    ulong boot_device; // if flags[1], drive, part1,2,3
    ulong cmdline;    // if flags[2], addr of cmd line
    ulong mods_count; // if flags[3], #of boot modules
    ulong mods_addr;  // if flags[3], addr of first
                     // boot module.
    union
    {
        aout_symbol_table_t aout_sym; // if flags[4], symbol table
                                     // from a.out kernel image
        elf_section_header_table_t elf_sec; // if flags[5], header
                                     // from ELF kernel.
    } u;
    ulong mmap_length; // if flags[6], BIOS mem map len
    ulong mmap_addr;  // if flags[6], BIOS map addr
    ulong drives_length; // if flags[7], BIOS drive info structs
    ulong drives_length; // if flags[7], first BIOS drive info
                     // struct.
    ulong config_table // if flags[8], ROM config table
    ulong boot_loader_name // if flags[9], addr of string
    ulong apm_table // if flags[10], addr of APM info table
    ulong vbe_control_info // if flags[11], video mode settings
    ulong vbe_mode_info
    ulong vbe_mode
    ulong vbe_interface_seg
    ulong vbe_interface_off
    ulong vbe_interface_len
};
-----
```

将控制权交给操作系统后，指向该结构体的指针将被存入寄存器 EBX。第一个字段 flags 表明下面哪几个域有效，没有使用的字段必须置为 0。详情可参见 www.gnu.org/software/grub/manual/multiboot/multiboot.html。

8.2.2 LILO

LILO (Linux LOader, Linux 加载程序) 多年来一直用做 x86 中 Linux 的加载程序。它是最早的引导加载程序之一, 用于帮助配置和加载 Linux 内核。LILO 也是一个两阶段的引导加载程序, 从这个角度而言, 它和 GRUB 非常相似, 但 LILO 仅使用配置文件, 并且没有命令行接口。

我们同样从 BIOS 初始化系统并将 MBR (第一阶段) 加载到内存, 然后将控制权交给第一阶段开始探索。LILO 运行时每一阶段的详细步骤如下。

第一阶段:

- (1) 开始执行并显示 “L”;
- (2) 检测磁盘几何信息并显示 “l.”;
- (3) 加载第二阶段的代码。

第二阶段:

- (1) 开始执行并显示 “L”;
- (2) 定位引导数据和操作系统, 并显示 “O.”;
- (3) 确定启动哪个操作系统, 并跳转到该操作系统。

LILO 配置文件中的一段内容如下:

```
-----
/etc/lilo.conf
image=/boot/bzImage-2.6.7-mytestkernel
label=Kernel 2.6.7, my test kernel
root=/dev/hda6
read-only
-----
```

包括以下参数: image, 指明内核所在的路径名; label, 用于描述配置的字符串; root, 指明根文件系统所驻留的分区; read-only, 表明根分区在引导时不可被修改。

以下是 GRUB 与 LILO 之间的区别。

- ❑ LILO 将配置信息存储在 MBR 中。若有任何改动, 必须运行 /sbin/lilo 来更新 MBR。
- ❑ LILO 不能读取不同的文件系统。
- ❑ LILO 没有交互式的命令行接口。

让我们回顾一下使用 LILO 作为引导加载程序时会发生什么。首先, MBR (包含 LILO) 将被复制到 0x7c00 处并开始执行。然后, LILO 开始工作, 它将从硬盘复制/etc/lilo.conf 中引用的内核映像。该映像 (由 build.c 创建) 由 init 扇区 (加载到 0x90000 处)、设置扇区 (加载到 0x90200 处) 和压缩映像 (加载到 0x10000 处) 组成。最后, LILO 将跳转到地址 0x90200 处的标号 start_of_setup 处。

8.2.3 PowerPC 和 Yaboot

Yaboot 是基于新世界 PowerPC 机器的 OF 的引导加载程序。Yaboot 与 LILO 和 GRUB 类似, 它使用配置文件和 ybin 或 ybootconfig 之类的实用程序来设置包含 Yaboot 的引导分区。与 x86 的 BIOS 类似, OF 允许配置引导设备, 但不同的系统, 配置不同。通常, 只要输入 “Command+Option/Alt+o+f.?” 就可以获得 OF 的设置。

Yaboot 引导时的步骤如下。

- (1) OF 调用 Yaboot。
- (2) 找到引导设备和引导路径，并打开引导分区。
- (3) 打开/etc/yaboot.conf 或命令解释器。
- (4) 加载映像或内核以及 initrd。
- (5) 执行映像。

如下所示，Yaboot 中内核加载的这段代码与 LILO 和 GRUB 相似：

```
-----  
yaboot.conf  
label=Linux  
root=/dev/hda11  
sysmap=/boot/System.map  
read-only  
-----
```

与 LILO 中一样，ybin 将 Yaboot 安装到引导分区。更新/修改 Yaboot 的配置后需要重新运行 ybin。

有关 Yaboot 的文档可在 www.penguinppc.org 上找到。

8.3 与体系结构相关的内存初始化

下面将详细介绍 PPC 和 x86 的硬件管理特征。x86 和 PowerPC 在硬件方面都具有支持实寻址和虚寻址的内存管理特征。和其他所有操作系统一样，Linux 内存管理也依赖于底层的硬件结构。本节描述 x86 和 PowerPC 体系结构的硬件初始化。由于内存管理的初始化与硬件息息相关，要理解其初始化过程就必须了解硬件的规格。与体系结构密切相关的天性决定了内存管理是首先进行初始化的子系统之一，并在 `start_kernel()` 执行前就开始被处理。

8.3.1 PowerPC 的硬件内存管理

本节描述 PowerPC 体系结构中硬件支持的地址转换，在 PowerPC 中也被称为“存储控制”。同时也分析 Linux 在从系统加电到内核初始化的过程中如何使用（或因可移植性的缘故而忽略）这些特征。

1. 实寻址模式

无论是嵌入式系统还是高性能系统，所有 PowerPC 的处理器都源于在实模式 (real mode)^① 下复位的硬件。PowerPC 的实寻址模式定义为禁止地址转换的处理器状态。地址转换由 MSR (Machine State Register, 机器状态寄存器) 中的 IR (instruction relocate, 指令重定位) 位和 DR (data relocate, 数据重定位) 位来控制。取指令时，如果 IR 位为 0，那么 EA (effective address, 有效地址) 就是实地址。加载和存储指令时，DR 位在 MSR 中也扮演类似的角色。

图 8-3 解释了 MSR，这是一个 64 位或 32 位寄存器，描述处理器的当前状态。在 32 位 MSR 中，IR 和 DR 分别是第 26 位和第 27 位。

^① 甚至技术上没有实模式的 440 系列处理器也是从“镜像”TLB 开始，该镜像将线性地址映射到物理地址。

图 8-3 PowerPC 的 MSR

下一级地址转换由段寄存器 (Segment Register) 中的二进制位 T 来决定。在 PowerPC 7xx 系列中, 有效地址中的第 0 位~第 3 位用于从 16 个段寄存器 (SR) 中选择一个。图 8-5 描述了段

寄存器。

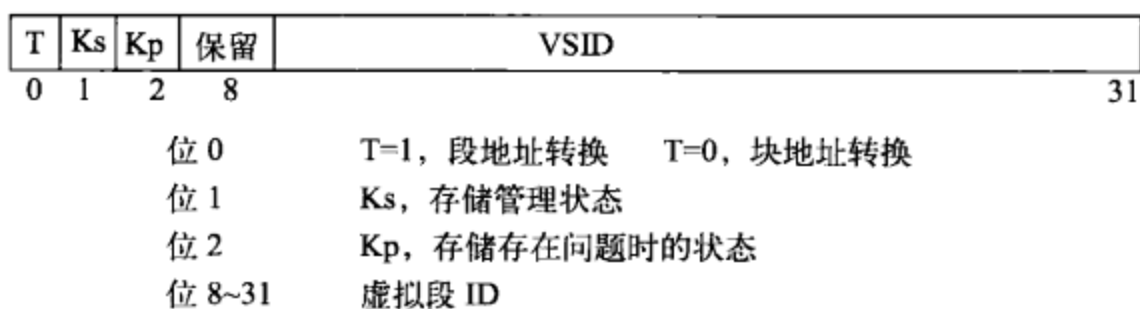


图 8-5 段寄存器

T 置位时, 该段就是 I/O 设备的**直接存储段** (direct store segment), 并且无需引用硬件页表。I/O 地址由权限位、BUID、与控制器相关的字段以及 EA 的第 4 位~第 31 位组成。Linux 没有使用直接存储段。

如果分段地址转换的普通段 T 未置位, 则使用 VSID (virtual segment ID, 虚拟段 ID)。

图 8-6 解释了一个 52 位 VA (virtual address, 虚拟地址) 是如何形成的。该地址由三个部分连接而成, 即有效地址的第 20 位~第 31 位 (给定页中的偏移量)、第 4 位~第 19 位和所选段寄存器 VSID 的第 8 位~第 31 位。VA 的最高 40 位组成了 VPN (virtual page number, 虚拟页号)。PowerPC 体系结构使用散列页表将虚拟页号映射到实际页面号 (即所需页在内存的实际地址)。散列函数通过使用虚拟页号和 SDR1 (Storage Description Register 1, 存储描述寄存器 1) 的值来存储和检索 PTE (Page Table Entry, 页表项)。PTE 是一个 8 字节的数据结构, 它包含内存页面所有必要的属性, 如图 8-7 所示。

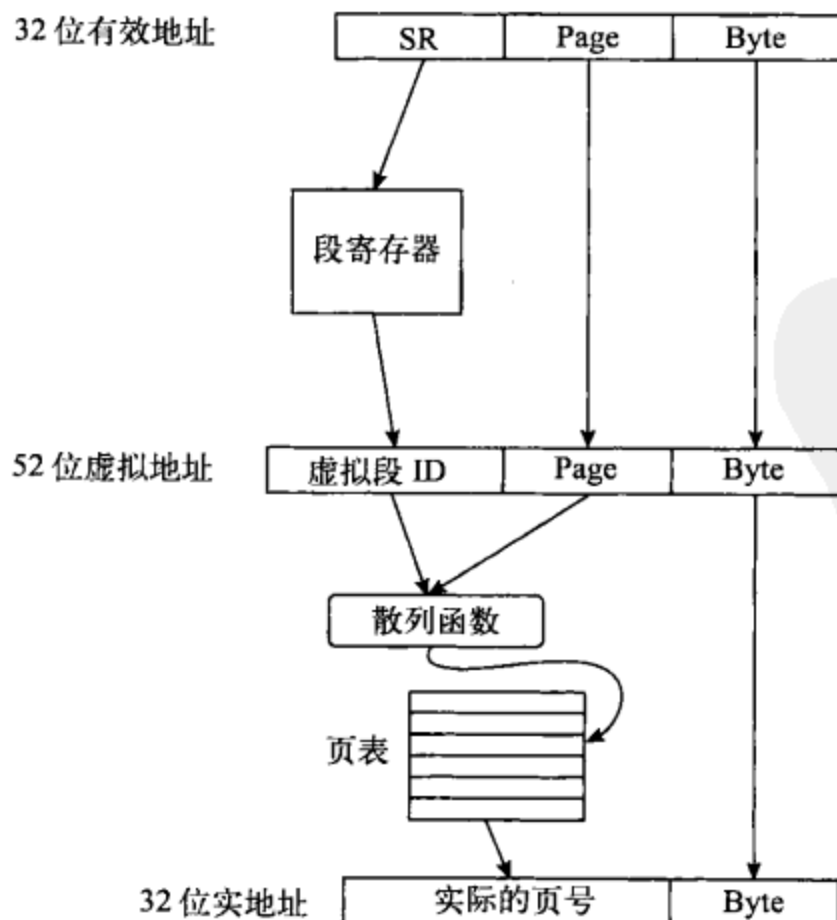


图 8-6 分段地址转换

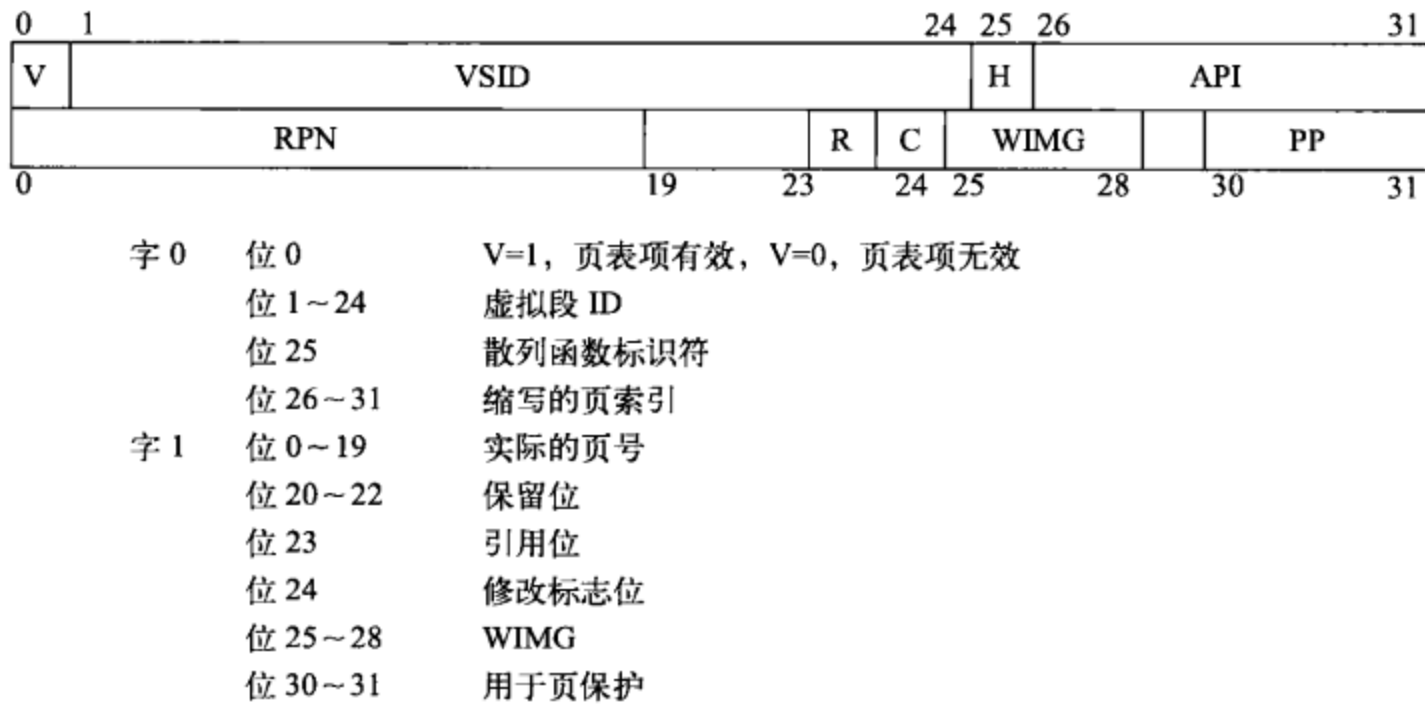


图 8-7 页表项

● 块地址转换

顾名思义, BAT (Block Address Translation, 块地址转换) 是允许映射 128 KB~256 MB 不等的连续内存块的寻址机制。在 PowerPC 体系结构中, BAT 寄存器是具有特权的 SPRs (special purpose registers, 专用寄存器), 如图 8-8 所示。

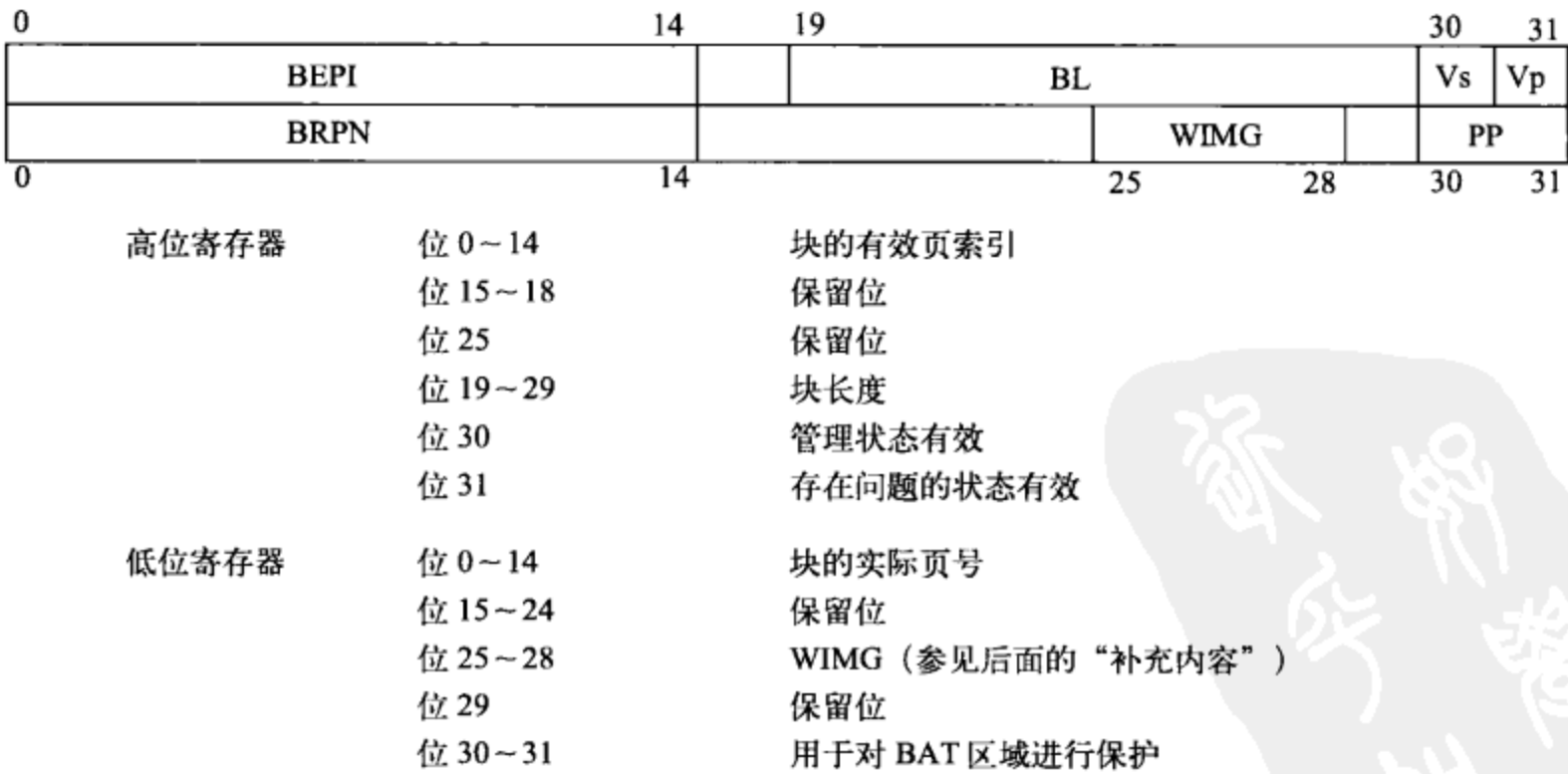


图 8-8 BAT 寄存器

BAT 寄存器中的信息如何形成实地址可参考图 8-9。通过 PPC 指令^①mtspr 和 mfspr 可

① 并不是所有 PowerPC 的处理器都实现了块地址转换。要注意的是, G4 和 G5 都没有实现块地址转换, 但已经在 4xx-嵌入式处理器中实现了。

以读写 4 个 IBAT (Instruction BAT, 指令块地址转换寄存器) 和 4 个 DBAT (Data BAT, 数据块地址转换寄存器)。

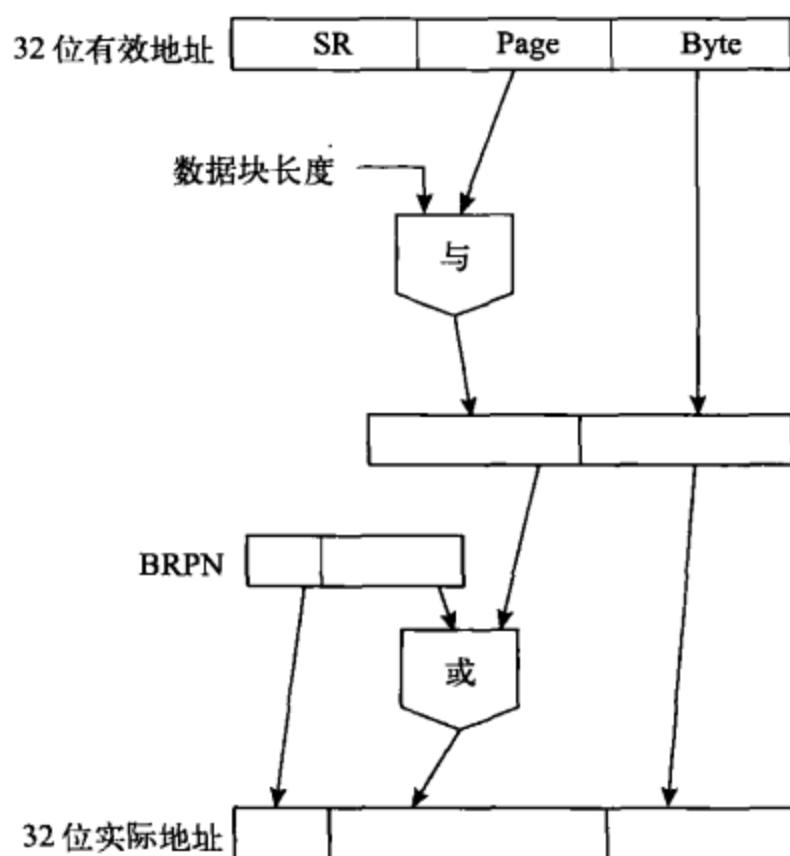


图 8-9 真正的 BAT

● 转换后备缓冲区

转换后备缓冲区 (Translation Lookaside Buffers, TLBs) 简称快表, 可以看做是硬件高速缓存, 它包含内存分页系统的硬件保护部分。TLB 的长度随着 PowerPC 体系结构的变化而有所不同, 它包含最近使用的页表项索引, 因而内存分页软件必须确保 TLB 和页表保持同步。当处理器在散列表^①中找不到某个页面时就会搜索 Linux 的页表, 如果还是没找到, 就会产生正常的缺页异常。有关 Linux 页表和 PPC 散列表之间同步的优化信息, 可参阅 Paul Mackerras 撰写的文档 “Low Level Optimizations in the PowerPC/Linux Kernels” (PowerPC/Linux 内核中的初级优化)。

● 存储访问模式控制 (Storage Access Mode Control)

当地址转换被启用 ($MSR_{IR}=1$, 或 $MSR_{DR}=1$, 或两者均为 1) 并借助分段地址转换或块地址转换来实现时, 存储方式由 4 个控制位来决定: W、I、M 和 G。对分段地址转换而言, 分别是页表项第二个字的第 25 位~第 28 位, 以及数据块地址转换寄存器 (DBAT) 第二个 SPR 的相同位。(G 位保存在指令块地址转换寄存器 IBAT 中。) 此外, 存放在页表项中的引用位和控制位也可用于分段地址转换。R 位和 C 位可由硬件或软件来设置。(有关二进制位 W、I、M、G、R 和 C 的讨论可参阅下面的补充内容)。

① 散列表并未在所有 PowerPC 处理器中实现。4xx-和 8xx- 嵌入式系统都没有实现散列表, 没有 TLB 时将产生一个硬件和内存分页软件的异常, 之后就会调入所请求的页面。

控制位

二进制位 W、I、M、G、R 和 C 决定处理器如何访问高速缓存和主存。

- ❑ W (Write Through)。若数据存储在高速缓存中并执行了存储操作，当 W=1 时，主存中的备份也必须更新。
- ❑ I (Cache Inhibit)。更新旁路高速缓存并直接访问主存。
- ❑ M (Memory Coherence)。当 M=1 时，强制硬件内存保持一致。
- ❑ G (Guard)。当 G=1 时，禁止使用推测性执行。
- ❑ R (Referenced)。当 R=1 时，页表项被引用。
- ❑ C (Changed)。当 C=1 时，页表项已被修改。

3. Linux 如何使用 PowerPC 的地址转换

让我们来看看 PPC 中哪些影响内存管理的代码。

下面是内核中最先获得控制权的代码。该例程回调 Firmware，并使用 claim() 函数来分配临时空间，之后内核将被解压到适当的位置：

```
-----
arch/ppc/boot/openfirmware/newworldmain.c
40 void boot(int a1, int a2, void *prom)
...
54 claim(initrd_start, RAM_END - initrd_start, 0);
55 printf("initial ramdisk moving 0x%x <- 0x%p (%x bytes)\n\r",
56   initrd_start, (char *)(&__ramdisk_begin), initrd_size);
57 memcpy((char *)initrd_start, (char *)(&__ramdisk_begin), initrd_size);
...
63 /* claim 3MB starting at PROG_START */
64 claim(PROG_START, PROG_SIZE, 0);
65 dst = (void *) PROG_START;
66 if (im[0] == 0x1f && im[1] == 0x8b) {
67   /* claim some memory for scratch space */
68   avail_ram = (char *) claim(0, SCRATCH_SIZE, 0x10);
69   begin_avail = avail_high = avail_ram;
70   end_avail = avail_ram + SCRATCH_SIZE;
71   printf("heap at 0x%p\n", avail_ram);
72   printf("gunzipping (0x%p <- 0x%p:0x%p)...", dst, im, im+len);
73   gunzip(dst, PROG_SIZE, im, &len);
74   printf("done %u bytes\n", len);
75   printf("%u bytes of heap consumed, max in use %u\n",
76     avail_high - begin_avail, heap_max);
...
86 sa = (unsigned long)PROG_START;
87 printf("start address = 0x%x\n", sa);
88
89 (*(kernel_start_t)sa)(a1, a2, prom);
-----
```

第 40 行：该文件的入口点是函数 boot(a1, a2, *prom)。

第 54 行：调用函数 claim() 分配 1 M 以下的内存，ramdisk 的内容也被复制到该内存中。

第 64 行：调用函数 `claim()` 为内核映像分配从 `0x1_0000` 开始的 3 M 内存。

第 68 行：调用函数 `claim()` 为 `scratch/heap` 分配从 `0x00` 开始的 8 K 内存。

第 73 行：内核映像被解压到地址 `0x1_0000` (`PROG_START`) 处。

第 89 行：使用参数 (`a1`、`a2` 和 `prom`) 跳转到 `0x1_0000` (`(*kernel_start_t)sa`) 处，此时 `a1` 将值存储在寄存器 `r3` 中 (等于导入 `ramdisk` 的起始地址)，`a2` 的值将存储在寄存器 `r4` 中 (等于导入 `ramdisk` 的大小，如果没有 `ramdisk`，则大小等于 `0xdeadbeef`)，`prom` 的值将存储在寄存器 `r5` 中 (代码存放在系统 ROM 中)。

下列代码块为各种 PowerPC 处理器的硬件内存管理功能做好了准备。RAM 中前 16 M 的内容将被映射到 `0xc0000000` 处：

```
-----
arch/ppc/kernel/head.S
131  __start:
...
150  bl  early_init  in <arch/ppc/kernel/setup.c> (283)
...
170  bl  mmu_off
...
171  RFI: SRR0=>IP, SRR1=>MSR
172  #ifndef CONFIG_POWER4
173  bl  clear_bats
174  bl  flush_tlbs
175
176  bl  initial_bats
177  #if !defined(CONFIG_APUS) && defined(CONFIG_BOOTX_TEXT)
178  bl  setup_disp_bat
179  #endif
180  #else /* CONFIG_POWER4 */
181  bl  reloc_offset
182  bl  initial_mm_power4
183  #endif /* CONFIG_POWER4 */
185  /*
186  * Call setup_cpu for CPU 0 and initialize 6xx Idle
187  */
188  bl  reloc_offset
189  li  r24,0      /* cpu# */
190  bl  call_setup_cpu /* Call setup_cpu for this CPU */
195  #ifdef CONFIG_POWER4
196  bl  reloc_offset
197  bl  init_idle_power4
198  #endif /* CONFIG_POWER4 */
199
210  bl  reloc_offset
211  mr  r26,r3
212  addis r4,r3,KERNELBASE@h /* current address of _start */
213  cmpwi 0,r4,0 /* are we already running at 0? */
214  bne  relocate_kernel
215
...
224  turn_on_mmu:
```

```

225 mfmsr r0
226 ori r0,r0,MSR_DR|MSR_IR
227 mtspr SRR1,r0
228 lis r0,start_here@h
229 ori r0,r0,start_here@l
230 mtspr SRR0,r0
231 SYNC
232 RFI /* enables MMU */

```

第 131 行：这是该代码的入口点。它将获得最小的 mmu 环境设置。（注意，APUS 代表 Amiga 加电系统。）

第 150 行：内核被加载的位置和它被链接的位置可能会有所不同。函数 `early_init` 用于返回当前代码的物理地址。

第 170 行：关闭 PPC 的内存管理单元。如果 IR 和 DR 都被置位则保持其状态，否则禁止重定位。

第 173~176 行：如果没有 power4 和 G5，则清除 BAT 寄存器，刷新 TLBs，并设置 BAT，以便将 RAM 的前 16 M 空间映射到 0xc0000000 处。

要注意的是，整个内核中所使用的各种内核内存标号：

```

-----
arch/ppc/defconfig
CONFIG_KERNEL_START=0xc0000000
-----

```

和

```

-----
include/asm-ppc/page.h
#define PAGE_OFFSET CONFIG_KERNEL_START
#define KERNELBASE PAGE_OFFSET
-----

```

第 181~182 行：通过使用分段来为 power4 和 G5 设置内核内存。

第 188~198 行：`setup_cpu()` 用于初始化内核和用户特征，例如高速缓存配置，以及是否存在 FPU 和 MMU。（注意，直到本书编写时，`init_idle_power4` 还没有实现任何操作。）

第 210 行：根据不同平台将内核重定位到 `KERNELBASE` 或 0x00 处。

第 224~232 行：通过将 MSR 中的 IR 和 DR 置位来启动 MMU（若 MMU 尚未准备就绪）。然后，执行 RFI 指令，跳转到标号 `label_here:` 处（注意，RFI 指令加载包含 SRR1 内容的 MSR，并跳转到 SRR0 所指的地址）。

以下是内核开始运行的代码。它基于命令行来设置系统中的所有内存：

```

-----
arch/ppc/kernel/head.S
1337 start_here:
...
1364 bl machine_init
1365 bl MMU_init
...

```

```

1385 lis r4,2f@h
1386 ori r4,r4,2f@l
1387 tophys(r4,r4)
1388 li r3,MSR_KERNEL & ~(MSR_IR|MSR_DR)
1389 FIX_SRR1(r3,r5)
1390 mtspr SRR0,r4
1391 mtspr SRR1,r3
1392 SYNC
1393 RFI
1394 /* Load up the kernel context */
1395 2: bl load_up_mmu
...
1411 /* Now turn on the MMU for real! */
1412 li r4,MSR_KERNEL
1413 FIX_SRR1(r4,r5)
1414 lis r3,start_kernel@h
1415 ori r3,r3,start_kernel@l
1416 mtspr SRR0,r3
1417 mtspr SRR1,r4
1418 SYNC
1419 RFI

```

第 1337 行：该行是这部分的入口点。

第 1364 行：machine_init()（参见文件 arch/ppc/kernel/setup.c，第 532 行）用于设置与机器相关的信息，例如 NVRAM、L2、CPU 高速缓存的行容量（CPU cache line size，即 CPU 每次与内存交换信息的单位量）以及调试信息，等等。

第 1365 行：MMU_init()（参见文件 arch/ppc/mm/init.c，第 234 行）根据 highmem 和 lowmem 得到总的内存大小，然后初始化内存管理单元的硬件（参见 MMU_init_hw()，第 267 行）、设置散列页表（参见 arch/ppc/mm/hashtable.s）、从 KERNELBASE（参见 mapin_ram()，第 272 行）开始映射所有 RAM 和 I/O（参见 setup_io_mappings()，第 285 行），并初始化上下文管理（参见 mmu_context_init()，第 288 行）。

第 1385 行：关闭 IR 和 DR，设置 SDR1。这样可以保存页表的实地址，还可以知道散列表中有多少位被用于页表索引。

第 1395 行：清除 TLBs，加载 SDR1（即散列表的基址和大小），设置分段，并根据特殊的 PowerPC 平台初始化块地址转换寄存器。

第 1412~1419 行：将 IR、DR 和 RFI 置位，以便启动/init/main.c 中的 start_kernel。注意，当 PowerPC 体系结构发生中断时，ISR（Instruction Address Register，指令地址寄存器）存储中断结束后寄存器返回的地址，该值被保存在 SRR0（Save Restore Register 0）中，相应地，机器状态寄存器被保存在 SRR1（Save Restore Register 1）中。简言之，发生中断时：

□ IAR->SRR0

□ MSR->SRR1

一般在中断例程的末尾将执行指令 RFI，它是上述过程的逆过程，此时用 SRR0 的值恢复 IAR，用 SRR1 恢复 MSR。简言之：

□ SRR0->IAR

□ SRR1->MSR

第 1385~1419 行的代码将通过以下 3 个步骤使用该方法来开启/关闭内存管理功能。

- (1) 在 SRR1 中为 MSR 设置所需的位 (参见图 8-1)。
- (2) 在 SRR0 中设置将要跳转到的地址。
- (3) 执行 RFI 指令。

8.3.2 基于 Intel x86 体系结构的硬件内存管理

加电后, 所有 Intel 处理器都工作在实地址模式下, 这是与早期 Intel 处理器兼容的一种寻址模式。现在, 处理器变得越来越复杂, 但某些遗留下来的代码也一直在使用, 新的处理器仍然必须支持这些代码才能运行。在实地址模式下, 处理器可以使用相同的指令来执行为 8086 和 8088 编写的程序, 更重要的是, 可以使用同一种寻址方式或地址转换 (address translation) 方式。地址转换的最终结果就说明了处理器是如何访问系统内存的。早期的 Intel 处理器有 20 位地址总线, 可以访问大约 64 KB 的内存, 这大大限制了系统中早期代码的大小。在实地址模式下, 线性地址就是物理地址。读者浏览初始化内存管理的代码时, 可以看到近年来处理器硬件方面的更多特性, 以及添加到软件中的更多复杂的数据结构。

setup.S 中执行关于内存初始化的几个重要函数的代码如下:

```
-----
arch/i386/boot/setup.S
307 #define SMAP 0x534d4150
308
309 meme820:
310     xorl    %ebx, %ebx    # continuation counter
311     movw    $E820MAP, %di    # point into the whitelist
312     # so we can have the bios
313     # directly write into it.
314
315     jmp     jme820:
316     movl    $0x0000e820, %eax    # e820, upper word zeroed
317     movl    $SMAP, %edx    # ascii 'SMAP'
318     movl    $20, %ecx    # size of the e820rec
319     pushw   %ds    # data record.
320     popw    %es
321     int     $0x15    # make the call
322     jc      bail820    # fall to e801 if it fails
323
324     cmpl    $SMAP, %eax    # check the return is 'SMAP'
325     jne     bail820    # fall to e801 if it fails
326
327     ...
328
329     good820:
330     movb    (E820NR), %al    # up to 32 entries
331     cmpb    $E820MAX, %al
332     jnl     bail820
333
334     incb    (E820NR)
```

```

339  movw  %di, %ax
340  addw  $20, %ax
341  movw  %ax, %di
342  again820:
343  cmpl  $0, %ebx    # check to see if
344  jne  jmp820      # %ebx is set to EOF
345  bail820:
-----

```

第 307~345 行：考虑代码段时，首先来看一个 BIOS 调用（第 321 行）int 15h，此时 ax=0xe820。该调用返回 BIOS 能够识别的多种不同内存类型的地址和长度。这一简单内存映射表示基本池，可从该基本池获得所有的 Linux 内存页面。进一步研究这些代码可以发现，内存映射可通过 3 种方式获得：0xe820、0xe801 和 0x88。这三种方式都必须与现有的 BIOS 及其平台兼容。

```

-----
arch/i386/boot/setup.S
595  # Now we move the system to its rightful place ... but we check if we have
    a # big-kernel. In that case we *must* not move it ...
597  testb $LOADED_HIGH, %cs:loadflags
598  jz  do_move0    # .. then we have a normal low
599          # loaded zImage
600          # .. or else we have a high
601          # loaded bzImage
602  jmp  end_move    # ... and we skip moving
603
604  do_move0:
605  movw  $0x100, %ax    # start of destination segment
606  movw  %cs, %bp      # aka SETUPSEG
607  subw  $DELTA_INITSEG, %bp    # aka INITSEG
608  movw  %cs:start_sys_seg, %bx    # start of source segment
609  cld
610  do_move:
611  movw  %ax, %es      # destination segment
612  incb  %ah          # instead of add ax, #0x100
613  movw  %bx, %ds      # source segment
614  addw  $0x100, %bx
615  subw  %di, %di
616  subw  %si, %si
617  movw  $0x800, %cx
618  rep
619  movsw
620  cmpw  %bp, %bx      # assume start_sys_seg > 0x200,
621          # so we will perhaps read one
622          # page more than needed, but
623          # never overwrite INITSEG
624          # because destination is a
625          # minimum one page below source
626  jb  do_move
627
628  end_move:
-----

```

第 595~628 行：以下内核映像的代码由 build.c 创建并由 LILO 加载。这些代码由 init 扇区（在地址 0x9000 处）、setup 扇区（在地址 0x9200 处）和压缩映像组成。该映像最初被加载到地址 0x10000 处，如果该映像是 LARGE (>0x7FF) 则留在原地，否则将被迁移到地址 0x1000 处。

```
-----
arch/i386/boot/setup.S
723  # Try enabling A20 through the keyboard controller
724  #endif /* CONFIG_X86_VOYAGER */
725  a20_kbc:
726  call empty_8042
727
728  #ifndef CONFIG_X86_VOYAGER
729  call a20_test      # Just in case the BIOS worked
730  jnz a20_done      # but had a delayed reaction.
731 #endif
732
733  movb $0xD1, %al    # command write
734  outb %al, $0x64
735  call empty_8042
736
737  movb $0xDF, %al    # A20 on
738  outb %al, $0x60
739  call empty_8042
-----
```

如何在 Intel 实地址模式下形成 20 位物理地址

IBM PC 机中最初使用的是 Intel 8088 处理器，仅有 20 位地址线[0…19]。它最多允许系统访问内部 1M 加大约 64 KB 的内存空间 (0~0x10_FFEF)，但物理上（即在总线上）可寻址的最后 64 K 内存实际上就是实存的前 64 K！

在处理器内部，16 位段选择器和 16 位段内偏移量组合形成一个 20 位地址。段选择器左移 4 位再加上段内偏移量，就将地址扩展了 4 位。这些寄存器的和就是总线上的物理地址。

例如：

为了获得最高位地址，可以加载一个值为 0xFFFF 的段寄存器（CS、DS、ES，等等）和一个值为 0xFFFF 的索引寄存器（SI、DI，等等）。在处理器内部，段寄存器左移 4 位并加上偏移量。

0xFFFF 左移 4 位 = 0x0F_FFF0

加上偏移量 + 0x00_FFFF

内部和 = 0x10_FFEF

外部物理地址 = 0x00_FFEF

最终得到的物理地址仍然是一个段选择器（值为 0x0000）和一个偏移量（值为 0xFFEF）的组合（0000：FFEF）。

访问最高地址及其以上的地址将绕回低地址 0xFFEF 处的低端内存。为该处理器编写的某些程序将依赖于这个 20 位的地址回绕行为。Intel 286 及以后的处理器拥有更宽的地址总线，

它与实寻址模式结合起来保持向后兼容 8088 和 8086。实寻址模式并不考虑那些遗留下来并依赖于 20 位地址回绕的软件，但加入了 A20M# 信号引脚来模拟早期处理器的这一特性。该信号可以屏蔽允许再次访问低端内存的 A20 信号。

我们通过一个逻辑门来启用或禁用内存总线的 A20 信号。最初设计这个门时使用的是键盘控制器上额外的 I/O 信号，该信号由 I/O 端口 0x60 和 0x64 来控制。后来又开发了一种使用 I/O 端口 0x92 的“快门 A20” (Fast Gate A20) 方法，并将该端口设计到系统板上。自从所有 x86 处理器都可以在实地址模式下复位以来，引导代码变得更加灵活，也可以使用上述方法来启用特定地址线的 A20 信号。

第 723~739 行：这些代码能够巧妙地转换到早期的 Intel 处理器，但这也给内存管理带来了麻烦。

```
-----
arch/i386/boot/setup.S
790 # set up gdt and idt
791 lidt idt_48      # load idt with 0,0
792 xorl %eax, %eax   # Compute gdt_base
793 movw %ds, %ax     # (Convert %ds:gdt to a linear ptr)
794 shll $4, %eax
795 addl $gdt, %eax
796 movl %eax, (gdt_48+2)
797 lgdt gdt_48      # load gdt with whatever is
798                  # appropriate
...
981 gdt:
982 .fill GDT_ENTRY_BOOT_CS,8,0
983
984 .word 0xFFFF      # 4Gb - (0x100000*0x1000 = 4Gb)
985 .word 0           # base address = 0
986 .word 0x9A00      # code read/exec
987 .word 0x00CF      # granularity = 4096, 386
988                  # (+5th nibble of limit)
989
990 .word 0xFFFF      # 4Gb - (0x100000*0x1000 = 4Gb)
991 .word 0           # base address = 0
992 .word 0x9200      # data read/write
993 .word 0x00CF      # granularity = 4096, 386
994                  # (+5th nibble of limit)
995 gdt_end:
996 .align 4
997
998 .word 0           # alignment byte
999 idt_48:
1000 .word 0          # idt limit = 0
1001 .word 0, 0       # idt base = 0L
1002
```



```

1003     .word 0      # alignment byte
1004 gdt_48:
1005     .word gdt_end - gdt - 1    # gdt limit
1006     .word 0, 0      # gdt base (filled in later)
-----

```

第 790~797 行：临时 GDT 和 IDT 的结构体和数据都被编译到 setup.S 的末尾，这些表都以最简单的形式来实现。

第 981~1006 行：这些行是临时 GDT 编译到 setup.S 中的值。GDT 有一个代码描述符和一个数据描述符，每个描述符都代表从 0x00 开始的 4 G 内存空间。IDT 被初始化到 0x00 处，稍后给它赋值。

涉及 Intel 平台的内存管理时，进入保护模式是最重要的阶段之一。此时，硬件开始为操作系统构建虚拟地址空间。

保护模式

Intel 的内存管理方式被称为保护模式。“保护”指的是多个相互独立的分段地址空间，它们互不干扰。Intel 内存管理的另一半是内存分页和页面转换。系统程序员可以使用分段和分页的各种组合，但 Linux 使用的是扁平模式 (flat model)，几乎没有用到分段管理。在扁平模式中，每个进程可以使用全部 32 位的地址空间 (4 GB)。

```

-----
arch/i386/boot/setupS
830  movw  $1, %ax      # protected mode (PE) bit
831  lmsw  %ax          # This is it!
832  jmp   flush_instr
833
834  flush_instr:
835  xorw  %bx, %bx      # Flag to indicate a boot
836  xorl  %esi, %esi     # Pointer to real-mode code
837  movw  %cs, %si
838  subw  $DELTA_INITSEG, %si
839  shll  $4, %esi
-----

```

第 830~831 行：在机器状态字中设置 PE 位将进入保护模式，jmp 指令在保护模式中开始执行。

第 834~839 行：保存指向实模式代码的 32 位指针，用于解压缩和稍后在 startup_32() 中加载内核。

回顾一下，在实寻址模式中，是使用 16 位指令来执行代码，并利用 .code16 汇编指令来编译当前文件，这些指令强制使用实寻址模式，这也就是 *Intel Programmer's Reference* (《Intel 程序员手册》) 中所说的 16 位模块。从 16 位模块跳转到 32 位模块时，Intel 体系结构 (以及汇编程序魔数) 允许在 16 位模块中创建 32 位指令。

下面创建并执行 32 位跳转：

```

-----
arch/i386/boot/setup.S
841 # jump to startup_32 in arch/i386/kernel/head.S
842 #
843 # NOTE: For high loaded big kernels we need a
844 #   jmp 0x100000, __BOOT_CS
845 #
846 # but we haven't yet reloaded the CS register, so the default size
847 # of the target offset still is 16 bit.
848 # However, using an operand prefix (0x66), the CPU will properly
849 # take our 48 bit far pointer. (INTEL 80386 Programmer's Reference
850 # Manual, Mixing 16-bit and 32-bit code, page 16-6)
851
852 .byte 0x66, 0xea    # prefix + jmp-opcode
853 code32: .long 0x1000    # will be set to 0x100000
854         # for big kernels
855 .word __BOOT_CS
-----

```

第 852 行：这一行创建 32 位跳转指令。

执行该跳转后，系统将使用临时 GDT，此时，代码从文件 arch/i386/kernel/head.S 第 57 行的标号 startup_32 处开始就运行在 32 位保护模式下了。

保护模式

迄今为止，我们讨论的都是如何使 Intel 系统准备好，以便设置内存分页。跟踪 head.S 的代码时，可以看到初始化时需要做些什么事情，以及 Linux 如何使用基于 x86 保护模式的分页系统。在 main.c 中，内核启动之前，还需要执行的代码如下所示。有关多种地址转换模式，以及内存初始化和 Intel 处理器的相关设置的完整信息，可参阅 *Intel Architecture Software Developers Manual* (《Intel 体系结构软件开发手册》)，第 III 卷。

```

-----
arch/i386/kernel/head.S
057 ENTRY(startup_32)
058
059 /*
060 * Set segments to known values.
061 */
062 cld
063 lgdt boot_gdt_descr - __PAGE_OFFSET
064 movl $(__BOOT_DS), %eax
065 movl %eax, %ds
066 movl %eax, %es
067 movl %eax, %fs
068 movl %eax, %gs
069
070 /*
071 * Initialize page tables. This creates a PDE and a set of page
072 * tables, which are located immediately beyond _end. The variable
073 * init_pg_tables_end is set up to point to the first "safe" location.
074 */
-----

```

```
085 * Mappings are created both at virtual address 0 (identity mapping)
086 * and PAGE_OFFSET for up to _end+sizeof(page tables)+INIT_MAP_BEYOND_END.
087 *
088 * Warning: don't use %esi or the stack in this code. However, %esp
089 * can be used as a GPR if you really need it...
090 */
091 page_pde_offset = (__PAGE_OFFSET >> 20);
092
093 movl $(pg0 - __PAGE_OFFSET), %edi
094 movl $(swapper_pg_dir - __PAGE_OFFSET), %edx
095 movl $0x007, %eax /* 0x007 = PRESENT+RW+USER */
096 10:
097 leal 0x007(%edi),%ecx /* Create PDE entry */
098 movl %ecx,(%edx) /* Store identity PDE entry */
099 movl %ecx,page_pde_offset(%edx) /* Store kernel PDE entry */
100 addl $4,%edx
101 movl $1024, %ecx
102 11:
103 stosl
104 addl $0x1000,%eax
105 loop 11b
106 /* End condition: we must map up to and including INIT_MAP_BEYOND_END */
107 /* bytes beyond the end of our own page tables;the +0x007 is the attribute bits */
108 leal (INIT_MAP_BEYOND_END+0x007)(%edi),%ebp
109 cmpl %ebp,%eax
110 jb 10b
111 movl %edi,(init_pg_tables_end - __PAGE_OFFSET)
112
113 #ifdef CONFIG_SMP
...
156 3:
157 #endif /* CONFIG_SMP */
158
159 /*
160 * Enable paging
161 */
162 movl $swapper_pg_dir-__PAGE_OFFSET,%eax
163 movl %eax,%cr3 /* set the page table pointer.. */
164 movl %cr0,%eax
165 orl $0x80000000,%eax
166 movl %eax,%cr0 /* ..and set paging (PG) bit */
167 ljmp $__BOOT_CS,$1f /* Clear prefetch and normalize %eip */
168 1:
169 /* Set up the stack pointer */
170 lss stack_start,%esp
...
177 pushl $0
178 popfl
179
```

```

180 #ifdef CONFIG_SMP
181     andl %ebx,%ebx
182     jz 1f      /* Initial CPU cleans BSS */
183     jmp checkCPUtype
184 1:
185 #endif /* CONFIG_SMP */
186
187 /*
188  * start system 32-bit setup. We need to re-do some of the things done
189  * in 16-bit mode for the "real" operations.
190  */
191     call setup_idt
192
193 *
194 * Copy bootup parameters out of the way.
195 * Note: %esi still has the pointer to the real-mode data.
196 */
197     movl $boot_params,%edi
198     movl $(PARAM_SIZE/4),%ecx
199     cld
200     rep
201     movsl
202     movl boot_params+NEW_CL_POINTER,%esi
203     andl %esi,%esi
204     jnz 2f     # New command line protocol
205     cmpw $(OLD_CL_MAGIC),OLD_CL_MAGIC_ADDR
206     jne 1f
207     movzwl OLD_CL_OFFSET,%esi
208     addl $(OLD_CL_BASE_ADDR),%esi
209 2:
210     movl $saved_command_line,%edi
211     movl $(COMMAND_LINE_SIZE/4),%ecx
212     rep
213     movsl
214 1:
215     checkCPUtype:
216     ...
279     lgdt cpu_gdt_descr
280     lidt idt_descr
281     ...
303     call start_kernel
-----

```

第 57 行：这是内核代码进入 32 位保护模式的入口点。当前这些代码使用临时 GDT。

第 63 行：该代码用引导时 GDT 的基地址初始化 GDTR。这个引导时的 GDT 与 setup.S 中所用的临时 GDT 相同（从地址 0x00000000 处开始的 4 GB 代码和数据），并仅由该引导代码使用。

第 64~68 行：__BOOT_DS 的值为（其取值请参见/include/asm-i386/segment.h），用于初始化剩余的段寄存器。该值指向 GDT 末尾的第 24 个选择器（从 0 开始编号），稍后将设置该寄

寄存器。

第 91~111 行：在 `swapper_pg_dir` 中创建一个页目录项 (PDE)，该页目录项引用页表 (`pg0`)，`pg0` 一开始的内容全部为 0，创建第一个目录项之后将复制 `PAGE_OFFSET` (内核内存) 的目录项。

第 113~157 行：该代码块初始化第二个 (非引导) 处理器的页表。我们主要讨论的是引导处理器。

第 162~164 行：寄存器 `cr3` 是 x86 中硬件分页的入口点。该寄存器被初始化为指向页目录 (Page Directory) 的基址，本例中是 `swapper_pg_dir`。

第 165~168 行：设置引导处理器中寄存器 `cr0` 的 PG (分页) 位。PG 位可启用 x86 体系结构的分页机制。进入或退出分页模式时，需要改变 PG 位，以确保处理器中所有指令都串行执行，建议使用跳转指令来实现 (见第 167 行)。

第 170 行：将栈初始化为数据段的起始位置 (可参阅第 401~403 行)。

第 177~178 行：寄存器 `eflags` 是读/写系统寄存器，它包含中断状态、模式和许可权。清除该寄存器可以使用如下方法：将 0 压入栈，然后使用 `popfl` 指令直接将它弹出到该寄存器。

第 180~185 行：通用寄存器 `ebx` 存储标志信息，标明运行该代码的处理器究竟是不是引导处理器。我们将它当做引导处理器而跟踪这些代码，因而 `ebx` 被清 0，并跳转到调用 `setup_idt` 的地址处。

第 191 行：`setup_idt` 例程初始化 IDT (Interrupt Descriptor Table, 中断描述符表)，它的每个表项都指向一个虚拟句柄。在第 7 章中已经介绍了 IDT，表中是一些函数 (或处理程序)，当处理器需要立即执行紧急代码时就调用这些函数。

第 197~214 行：用户可以在引导时传递某些参数给 Linux，这些参数存储在此处，稍后将会用到。

第 215~303 行：这些代码为 x86 处理器做了大量乏味但必须的工作，即版本检测和一些次要的初始化。借助于 `cupid` 指令 (或者不用该指令) 可以设置寄存器 `eflags` 和寄存器 `cr0` 的某些位。`cr0` 中值得注意的是位 4，它代表扩展类型 (ET)。该位表明支持老式 x86 处理器中的数学协处理器指令。这块代码中最重要的是第 279~280 行，它们将 IDT 和 GDT (借助 `lidt` 和 `lgdt` 指令) 加载到寄存器 `idtr` 和 `gdtr`。最后，第 303 行，跳转到例程 `start_kernel()` 处。

系统使用 `head.S` 中的代码可以将逻辑地址映射到线性地址，最后映射到物理地址 (参见图 8-10)。从逻辑地址开始，段选择器 (CS、DS、ES 等) 引用 GDT 中的一个描述符，而偏移量就是找到的扁平地址。描述符和偏移量组合，就形成了逻辑地址。

代码遍查时，读者可以看到如何创建页目录 (`swapper_pg_dir`) 和页表 (`pg0`)，以及寄存器 `cr3` 如何初始化为指向页目录。如前所述，处理器根据寄存器 `cr3` 的设置就知道要到何处寻找内存分页的部件，并设置寄存器 `cr0` (PG 位)，以通知处理器可以开始使用这些部件了。逻辑地址的第 22~31 位表示 PDE，第 12~21 位表示 PTE，第 0~11 位表示物理页内的偏移量 (本例中是 4 KB)。

现在，该系统使用临时分页系统可以定制 8 MB 内存。下一步就是调用 `init/main.c` 中的函数 `start_kernel()`。

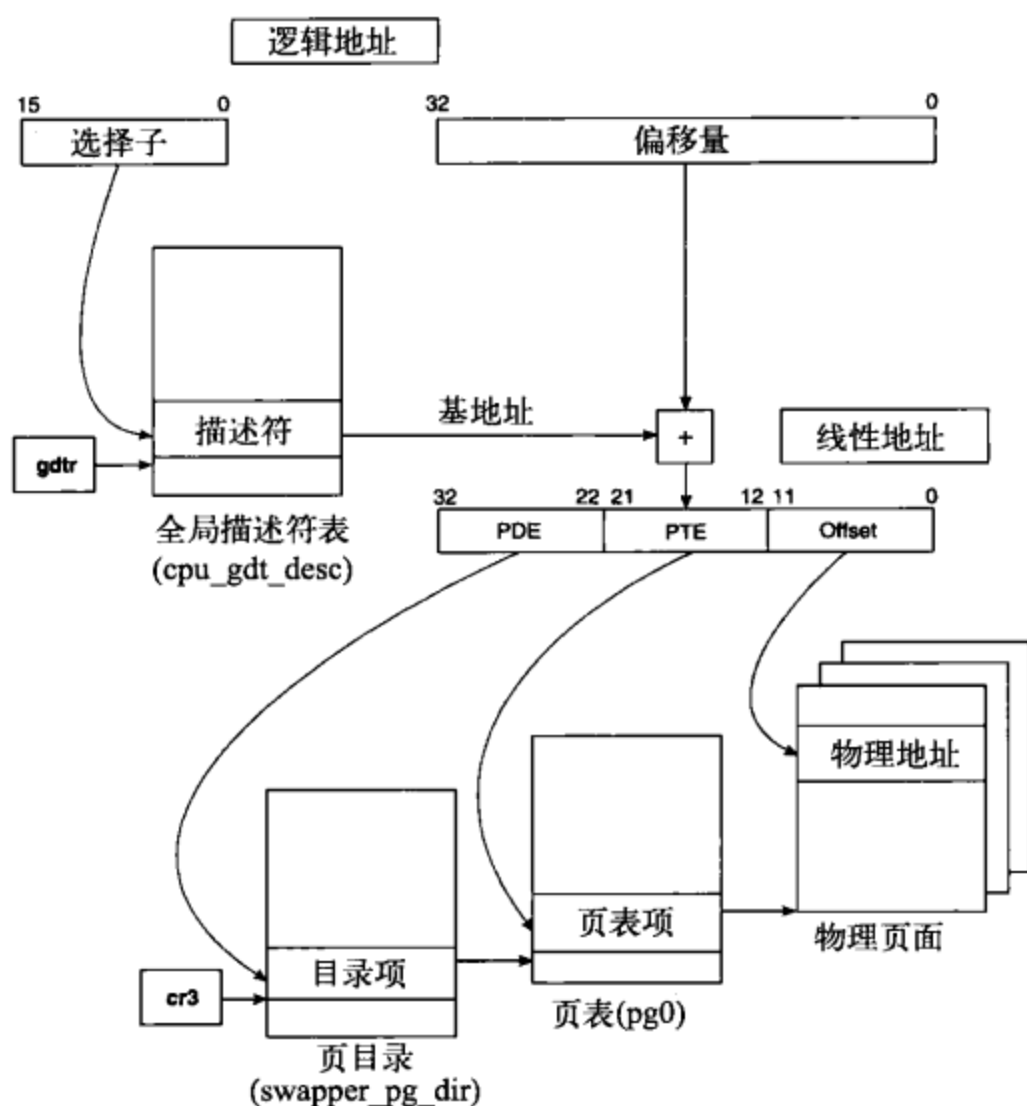


图 8-10 引导时的内存分页

8.3.3 PowerPC 和 x86 的代码汇集

注意：现在 PowerPC 和 x86 的代码都集中在 `init/main.c` 的 `start_kernel()` 中。该例程位于与体系结构无关的代码段中，将调用特定体系结构的例程来完成内存初始化。

该文件中调用的第一个函数就是 `arch/i386/kernel/setup.c` 中的 `setup_arch()`，再调用 `arch/i386/mm/init.c` 中的 `paging_init()`，之后 `paging_init()` 调用同一文件中的 `pagetable_init()`。系统中其余的内存在此处被分配出去，以生成最后的页表。

在 PowerPC 的世界里，很多事情都已经完成了。接下来，`arch/ppc/kernel/setup.c` 中的 `setup_arch()` 将调用 `arch/ppc/mm/init.c` 内的 `paging_init()`。值得注意的是，PPC 的 `paging_init()` 会将所有页面都设置到 DMA 区。

8.4 原始的 RAM 盘

LILO、GRUB 和 Yaboot 都可以加载原始的 RAM 盘 (`initrd`)。在加载并初始化最终的根文件系统之前，`initrd` 扮演根文件系统的角色。我们也把最终根文件系统的加载称为根的旋转。

该初始化步骤允许 Linux 一开始就加载某些预编译模块，然后动态加载其他模块和 `initrd` 的驱动程序。`initrd` 与引导加载程序的主要区别是，它在第二阶段时加载一个最小的内核和

RAM 盘。内核利用 RAM 盘来初始化并挂载最终的根文件系统，然后删除 initrd。

initrd 支持如下操作：

- 引导时配置内核；
- 保持一个小的通用内核；
- 有一个用于若干硬件配置的内核；

前面引用的章节是使用 Yaboot、GRUB 和 LILO 装载 Linux 时最常见的。每个引导加载程序都有丰富的用于配置文件的命令集。对于定制的或特殊的引导过程而言，可以在网上搜索到很多与 GRUB 和 LILO 配置文件相关的信息。

现在，我们已经讨论过如何加载内核以及如何启动内存的初始化过程，接下来就要探讨内核的初始化过程了。

8.5 开始：start_kernel()

我们从函数 start_kernel()（见 init/main.c）开始讨论，这是最先调用的与体系结构无关的代码。

跳转到 start_kernel()时，会执行进程 0，也就是通常所说的根用户线程（root thread）。进程 0 孕育了进程 1，也就是 init 进程，然后进程 0 就变成 CPU 的空闲进程。在调用/sbin/init 时，系统中仅有这两个进程在运行：

```
-----
init/main.c
396 asmlinkage void __init start_kernel(void)
397 {
398     char * command_line;
399     extern char saved_command_line[];
400     extern struct kernel_param __start__param[], __stop__param[];
...
405     lock_kernel();
406     page_address_init();
407     printk(linux_banner);
408     setup_arch(&command_line);
409     setup_per_cpu_areas();
...
415     smp_prepare_boot_cpu();
...
422     sched_init();
423
424     build_all_zonelists();
425     page_alloc_init();
426     printk("Kernel command line: %s\n", saved_command_line);
427     parse_args("Booting kernel", command_line, __start__param,
428         __stop__param - __start__param,
429         &unknown_bootoption);
430     sort_main_extable();
431     trap_init();
432     rcu_init();
433     init_IRQ();
434     pidhash_init();
```



```

435  init_timers();
436  softirq_init();
437  time_init();
...
444  console_init();
445  if (panic_later)
446    panic(panic_later, panic_param);
447  profile_init();
448  local_irq_enable();
449  #ifdef CONFIG_BLK_DEV_INITRD
450    if (initrd_start && !initrd_below_start_ok &&
451        initrd_start < min_low_pfn << PAGE_SHIFT) {
452      printk(KERN_CRIT "initrd overwritten (0x%08lx < 0x%08lx) - "
453        "disabling it.\n", initrd_start, min_low_pfn << PAGE_SHIFT);
454      initrd_start = 0;
455    }
456  #endif
457  mem_init();
458  kmem_cache_init();
459  if (late_time_init)
460    late_time_init();
461  calibrate_delay();
462  pidmap_init();
463  pgtable_cache_init();
464  prio_tree_init();
465  anon_vma_init();
466  #ifdef CONFIG_X86
467    if (efi_enabled)
468      efi_enter_virtual_mode();
469  #endif
470  fork_init(num_physpages);
471  proc_caches_init();
472  buffer_init();
473  unnamed_dev_init();
474  security_scaffolding_startup();
475  vfs_caches_init(num_physpages);
476  radix_tree_init();
477  signals_init();
478  /* rootfs populating might need page-writeback */
479  page_writeback_init();
480  #ifdef CONFIG_PROC_FS
481    proc_root_init();
482  #endif
483  check_bugs();
...
490  init_idle(current, smp_processor_id());
...
493  rest_init();
494  }

```

8.5.1 调用 lock_kernel()

第 405 行: 在 Linux 2.6 内核中, 默认的配置支持抢占式内核。抢占式内核意味着内核本身

可以被优先级更高的任务中断，如硬件中断，并由高优先级的任务获得控制权。因此，内核必须保存足够多的状态信息，以便高优先级的任务完成后能够返回并执行原来的任务。

早期版本的 Linux 利用 BKL (Big Kernel Lock, 大内核锁) 实现了内核抢占和 SMP (对称多处理器) 锁定。在后来的版本中，Linux 将抢占抽象成各种调用，比如 `preempt_disable()`。BKL 至今仍用于初始化过程，它是一个递归的自旋锁，可以被给定的 CPU 多次运行。使用 BKL 的副作用就是它禁止内核抢占，这对初始化过程非常不利。

锁住内核可以防止它被任何其他任务中断或抢占，Linux 是使用 BKL 来实现这一功能的。当内核被锁住时，其他任何进程都不能被执行，这与任何时候都可以中断的抢占式内核刚好相反。在 Linux 2.6 内核中，使用 BKL 在启动时锁住内核，并初始化各种内核对象而不用担心被中断。第 493 行的 `rest_init()` 函数将内核解锁。因此，`start_kernel()` 的所有工作都将在内核被锁住时完成。接下来让我们来看看 `lock_kernel()` 函数会做些什么：

```
-----
include/linux/smp_lock.h
42 static inline void lock_kernel(void)
43 {
44     int depth = current->lock_depth+1;
45     if (likely(!depth))
46         get_kernel_lock();
47     current->lock_depth = depth;
48 }
-----
```

第 44~48 行：init 任务有一个特殊的 `lock_depth`，其值为 -1，它确保在多处理器系统中，不同的 CPU 不会试图同时抢夺内核锁。由于仅有一个 CPU 运行 init 任务，且只有 init 的 `depth` 是 0（其他所有任务的 `depth` 都大于 0），因此，只有该任务可以抢夺大内核锁。函数 `unlock_kernel()` 中也采用了类似的技巧，可以在该函数中测试 (`--current->lock_depth<0`)。现在，让我们来看看函数 `get_kernel_lock()` 会做些什么：

```
-----
include/linux/smp_lock.h
10 extern spinlock_t kernel_flag;
11
12 #define kernel_locked()    (current->lock_depth >= 0)
13
14 #define get_kernel_lock()  spin_lock(&kernel_flag)
15 #define put_kernel_lock()  spin_unlock(&kernel_flag)
...
59 #define lock_kernel()      do { } while(0)
60 #define unlock_kernel()     do { } while(0)
61 #define release_kernel_lock(task) do { } while(0)
62 #define reacquire_kernel_lock(task) do { } while(0)
63 #define kernel_locked()    1
-----
```

第 10~15 行：这些宏描述那些使用标准自旋锁例程的大内核锁。在多处理器系统中，可能会有两个 CPU 试图访问同一个数据结构，在第 7 章阐述的自旋锁可以预防这种争夺。

第 59~63 行: 当内核不能被抢占且不会运行于多个 CPU 上时, 由于不会发生任何中断, 因而 lock_kernel() 什么都不做。

现在, 内核已经抓住了 BKL, 直到 start_kernel() 结束时才会释放, 因此, 以下所有命令都不能被抢占。

8.5.2 调用 page_address_init()

第 406 行: 在与体系结构相关部分的代码中, page_address_init() 是内存子系统初始化时最先调用的函数, 其定义根据编译时 3 个不同参数的值而定。前两个参数通过将函数体定义为 do{}while(0), 使得 page_address_init() 停下来什么都不做, 如下列代码所示。第三个参数将在此处详细探讨。让我们来看看这些不同的定义, 并讨论它们在什么情况下启用:

```
-----
include/linux/mm.h
376 #if defined(WANT_PAGE_VIRTUAL)
382 #define page_address_init() do { } while(0)

385 #if defined(HASHED_PAGE_VIRTUAL)
388 void page_address_init(void);

391 #if !defined(HASHED_PAGE_VIRTUAL) && !defined(WANT_PAGE_VIRTUAL)
394 #define page_address_init() do { } while(0)
-----
```

当系统允许直接映射内存时, 就设置 WANT_PAGE_VIRTUAL 的 #define, 此时仅计算出其虚拟地址就能够访问对应的存储单元了。万一所有 RAM 都无法映射到内核地址空间 (配置了高端内存 himem 后会经常发生这种情况), 就需要更多相关途径来获得内存地址。因此, 页面寻址的初始化只有在设置了 HASHED_PAGE_VIRTUAL 后才定义。

现在, 来考虑一下内核在何处需被告知使用了 HASHED_PAGE_VIRTUAL, 并在何处需要初始化内核正在使用的虚拟内存。记住, 这只有当配置了 himem 时才会发生, 就是说, 内核可以访问的 RAM 大小远比能被内核地址空间映射的 RAM 大小大得多 (通常是 4 GB)。

定义以下函数的过程中将会引入并反复访问各种内核对象。表 8-3 给出了分析函数 page_address_init() 引入到的内核对象。

表8-3 调用函数page_address_init()时引入的内核对象

对 象 名	描 述
page_address_map	结构体
page_address_slot	结构体
page_address_pool	全局变量
page_address_maps	全局变量
page_address_htable	全局变量

```
-----
mm/highmem.c
510 static struct page_address_slot {
511     struct list_head lh;
```

```

512 spinlock_t lock;
513 } ____cacheline_aligned_in_smp page_address_htable[1<<PA_HASH_ORDER];
...
591 static struct page_address_map page_address_maps[LAST_PKMAP];
592
593 void __init page_address_init(void)
594 {
595     int i;
596
597     INIT_LIST_HEAD(&page_address_pool);
598     for (i = 0; i < ARRAY_SIZE(page_address_maps); i++)
599         list_add(&page_address_maps[i].list, &page_address_pool);
600     for (i = 0; i < ARRAY_SIZE(page_address_htable); i++) {
601         INIT_LIST_HEAD(&page_address_htable[i].lh);
602         spin_lock_init(&page_address_htable[i].lock);
603     }
604     spin_lock_init(&pool_lock);
605 }

```

第 597 行：本行旨在初始化全局变量 `page_address_pool`，这是一个 `list_head` 类型的结构体，指向从 `page_address_maps`（第 591 行）分配的空闲页链表。图 8-11 解释了 `page_address_pool`。

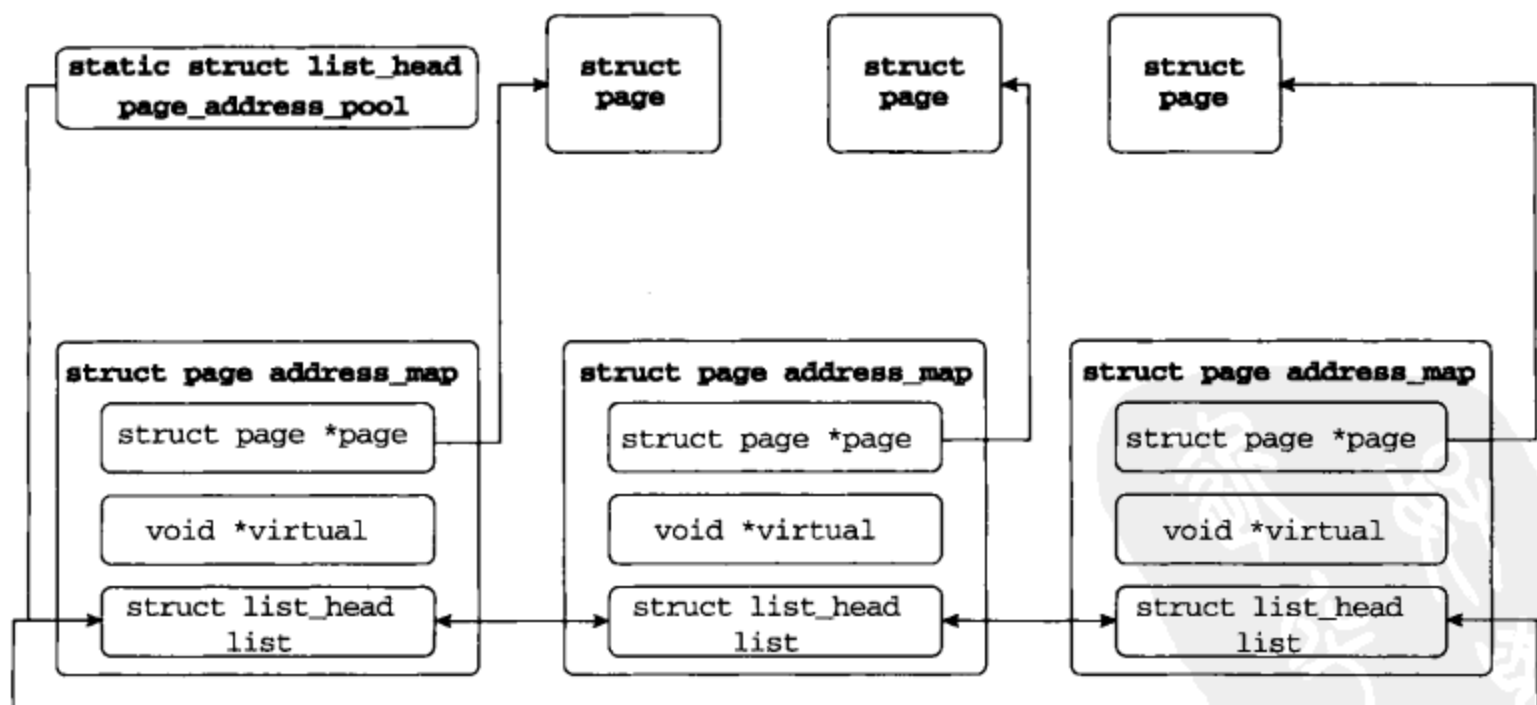


图 8-11 围绕页地址映射池的数据结构

第 598~599 行：将 `page_address_map` 的每个页链表加入到以 `page_address_pool` 为表头的双向链表中。稍后将详细讨论结构体 `page_address_map`。

第 600~603 行：初始化每个页地址散列表的 `list_head` 和自旋锁。散列到同一地址的元素形成的链表将存放在变量 `page_address_htable` 中。图 8-12 描述了页地址散列表。

第 604 行：初始化 `page_address_pool` 的自旋锁。

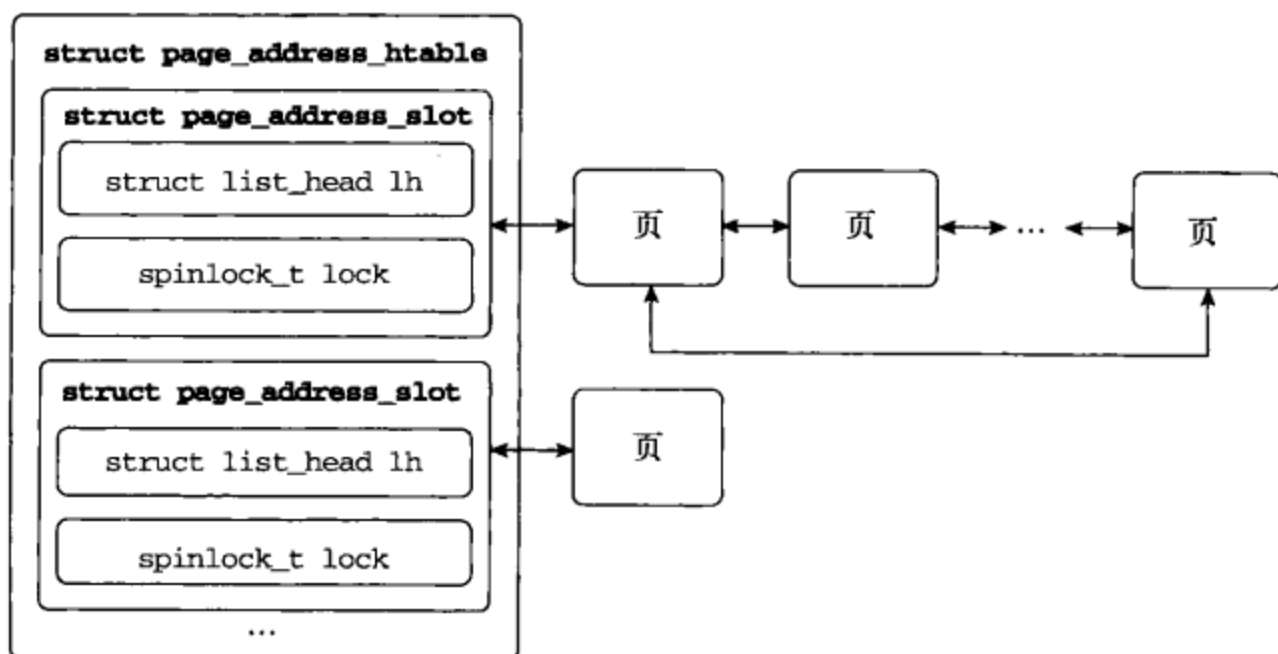


图 8-12 页地址散列表

下面, 分析一下结构体 `page_address_map`, 以便更好地理解刚才初始化过的列表。该结构体旨在保持页面及其虚拟地址之间的联系, 如果页面与其虚拟地址之间是一个线性关系的话就太浪费了。仅当使用散列寻址时该结构体才有必要存在:

```

-----
mm/highmem.c
490 struct page_address_map {
491     struct page *page;
492     void *virtual;
493     struct list_head list;
494 };
-----
  
```

可以看到, 该对象用于保存指向与该页相关的页结构的指针、指向虚拟地址的指针, 以及 `list_head` 结构体, 以便保持它在页面地址所形成的双向链表中的位置。

8.5.3 调用 `printk(linux_banner)`

第 407 行: 该调用负责 Linux 内核第一个控制台的输出。此处引入全局变量 `linux_banner`:

```

-----
init/version.c
31 const char *linux_banner =
32     "Linux version " UTS_RELEASE " (" LINUX_COMPILE_BY "@"
LINUX_COMPILE_HOST ") (" LINUX_COMPILER ") " UTS_VERSION "\n";
-----
  
```

如上例所示, 文件 `version.c` 中定义了 `linux_banner`, 该字符串将为用户提供内核版本, 用来编译该内核的 gcc 版本, 以及发布版本等参考信息。

8.5.4 调用 `setup_arch`

第 408 行: `arch/i386/kernel/setup.c` 中的函数 `setup_arch()` 被转换成 `__init` 类型 (有关 `__init`

的描述参见第2章), 并仅在系统初始化时运行一次。函数 `setup_arch()` 接收一个指向引导时输入的任何 Linux 命令行数据的指针, 并初始化许多特定于体系结构的子系统, 例如内存、I/O、处理器和控制台:

```
-----
arch/i386/kernel/setup.c
1083 void __init setup_arch(char **cmdline_p)
1084 {
1085     unsigned long max_low_pfn;
1086
1087     memcpy(&boot_cpu_data, &new_cpu_data, sizeof(new_cpu_data));
1088     pre_setup_arch_hook();
1089     early_cpu_init();
1090
1091     /*
1092     * FIXME: This isn't an official loader_type right
1093     * now but does currently work with elilo.
1094     * If we were configured as an EFI kernel, check to make
1095     * sure that we were loaded correctly from elilo and that
1096     * the system table is valid. If not, then initialize normally.
1097     */
1098     #ifdef CONFIG_EFI
1099     if ((LOADER_TYPE == 0x50) && EFI_SYSTAB)
1100         efi_enabled = 1;
1101     #endif
1102
1103     ROOT_DEV = old_decode_dev(ORIG_ROOT_DEV);
1104     drive_info = DRIVE_INFO;
1105     screen_info = SCREEN_INFO;
1106     edid_info = EDID_INFO;
1107     apm_info.bios = APM_BIOS_INFO;
1108     ist_info = IST_INFO;
1109     saved_videomode = VIDEO_MODE;
1110     if( SYS_DESC_TABLE.length != 0 ) {
1111         MCA_bus = SYS_DESC_TABLE.table[3] & 0x2;
1112         machine_id = SYS_DESC_TABLE.table[0];
1113         machine_submodel_id = SYS_DESC_TABLE.table[1];
1114         BIOS_revision = SYS_DESC_TABLE.table[2];
1115     }
1116     aux_device_present = AUX_DEVICE_INFO;
1117
1118     #ifdef CONFIG_BLK_DEV_RAM
1119     rd_image_start = RAMDISK_FLAGS & RAMDISK_IMAGE_START_MASK;
1120     rd_prompt = ((RAMDISK_FLAGS & RAMDISK_PROMPT_FLAG) != 0);
1121     rd_doload = ((RAMDISK_FLAGS & RAMDISK_LOAD_FLAG) != 0);
1122     #endif
1123     ARCH_SETUP
1124     if (efi_enabled)
1125         efi_init();
1126     else
1127         setup_memory_region();
1128
1129     copy_edd();
1130
```

```
1131 if (!MOUNT_ROOT_RDONLY)
1132     root_mountflags &= ~MS_RDONLY;
1133 init_mm.start_code = (unsigned long) _text;
1134 init_mm.end_code = (unsigned long) _etext;
1135 init_mm.end_data = (unsigned long) _edata;
1136 init_mm.brk = init_pg_tables_end + PAGE_OFFSET;
1137
1138 code_resource.start = virt_to_phys(_text);
1139 code_resource.end = virt_to_phys(_etext)-1;
1140 data_resource.start = virt_to_phys(_etext);
1141 data_resource.end = virt_to_phys(_edata)-1;
1142
1143 parse_cmdline_early(cmdline_p);
1144
1145 max_low_pfn = setup_memory();
1146
1147 /*
1148  * NOTE: before this point _nobody_ is allowed to allocate
1149  * any memory using the bootmem allocator.
1150  */
1151
1152 #ifdef CONFIG_SMP
1153     smp_alloc_memory(); /* AP processor realmode stacks in low memory*/
1154 #endif
1155 paging_init();
1156
1157 #ifdef CONFIG_EARLY_PRINTK
1158 {
1159     char *s = strstr(*cmdline_p, "earlyprintk=");
1160     if (s) {
1161         extern void setup_early_printk(char *);
1162
1163         setup_early_printk(s);
1164         printk("early console enabled\n");
1165     }
1166 }
1167 #endif
1168 ...
1170 dmi_scan_machine();
1171
1172 #ifdef CONFIG_X86_GENERICARCH
1173     generic_apic_probe(*cmdline_p);
1174 #endif
1175 if (efi_enabled)
1176     efi_map_memmap();
1177
1178 /*
1179  * Parse the ACPI tables for possible boot-time SMP configuration.
1180  */
1181 acpi_boot_init();
1182
1183 #ifdef CONFIG_X86_LOCAL_APIC
1184     if (smp_found_config)
1185         get_smp_config();
1186 #endif
```



```

1187
1188 register_memory(max_low_pfn);
1188
1190 #ifdef CONFIG_VT
1191 #if defined(CONFIG_VGA_CONSOLE)
1192     if (!efi_enabled || (efi_mem_type(0xa0000) != EFI_CONVENTIONAL_MEMORY))
1193         conswitchp = &vga_con;
1194 #elif defined(CONFIG_DUMMY_CONSOLE)
1195     conswitchp = &dummy_con;
1196 #endif
1197 #endif
1198 }

```

第 1087 行：获得 `boot_cpu_data` 的值，这是指向 `cpuinfo_x86` 结构体的指针，引导时给该结构体赋值。PPC 系统中与之类似。

第 1088 行：激活所有特定于硬件的身份识别例程。可查阅 `arch/xxx/machine-default/setup.c`。

第 1089 行：识别特定的处理器。

第 1103~1116 行：获取系统的引导参数。

第 1118~1122 行：如果在 `arch/<arch>/defconfig` 中有相应设置，则获取 RAM 盘。

第 1124~1127 行：初始化可扩展固件接口 (Extensible Firmware Interface) (如果在 `defconfig` 中有相应设置的话)，或仅输出 BIOS 内存映像。

第 1129 行：保存引导时增强型磁盘驱动器的参数。

第 1133~1141 行：初始化由 BIOS 提供的内存映像中的内存管理结构。

第 1143 行：开始分析 Linux 命令行 (详情可查阅 `arch/<arch>/kernel/setup.c`)。

第 1145 行：初始化/保留引导内存 (详情可查阅 `arch/i386/kernel/setup.c`)。

第 1153~1155 行：获取 SMP 初始化时所需的页面，或初始化 8 M 以上的内存分页，8 M 以内的页面已经在 `head.S` 中被初始化 (详情可查阅 `arch/i386/mm/init.c`)。

第 1157~1167 行：让 `printk()` 运行，即使此时尚未完成控制台的所有初始化过程也是如此。

第 1170 行：DMI (Desktop Management Interface, 桌面管理界面)，用于从 BIOS 收集特定系统硬件的配置信息 (可查阅 `arch/i386/kernel/dmi_scan.c`)。

第 1172~1174 行：如果配置程序调用该函数，则寻找由命令行给出的 APIC (详情可查阅 `arch/i386/machine-generic/probe.c`)。

第 1175~1176 行：如果使用可扩展固件接口，则重新映射 EFI 内存映像 (参见 `arch/i386/kernel/efi.c`)。

第 1181 行：寻找局部 APIC 和 I/O APIC。(参见 `arch/i386/kernel/acpi/boot.c`) 定位并检验系统描述符表 (参见 `drivers/acpi/tables.c`)。想要更好地理解 ACPI 的话，可以在网上查阅 ACPI4LINUX 项目。

第 1183~1186 行：扫描 SMP 的配置 (参见 `arch/i386/kernel/mpparse.c`)。本节也可以将 ACPI 用做配置信息。

第 1188 行：为标准资源请求 I/O 和内存空间。(如果想了解如何注册资源，可查阅 `arch/i386/`

kernel/std_resources.c。)

第 1190~1197 行: 设置 VGA 控制台转换结构。(参见 drivers/video/console/vgacon.c)

PowerPC 中一个类似于 setup_arch(), 但更简练的函数位于文件 arch/ppc/kernel/setup.c 中。结构体 ppc_md 的大部分内容由该函数来初始化。调用 arch/ppc/platforms/pmac_feature.c 中的函数 pmac_feature_init() 进行初步检测并初始化 pmac 硬件。

8.5.5 调用 setup_per_cpu_areas()

第 409 行: 例程 setup_per_cpu_areas() 用于设置多处理器环境。如果 Linux 内核编译时不支持 SMP, 那么该函数什么都不做, 如下所示:

```
-----
init/main.c
317 static inline void setup_per_cpu_areas(void) { }
-----
```

如果 Linux 内核编译时支持 SMP, 那么该函数定义如下:

```
-----
init/main.c
327 static void __init setup_per_cpu_areas(void)
328 {
329     unsigned long size, i;
330     char *ptr;
331     /* Created by linker magic */
332     extern char __per_cpu_start[], __per_cpu_end[];
333
334     /* Copy section for each CPU (we discard the original) */
335     size = ALIGN(__per_cpu_end - __per_cpu_start, SMP_CACHE_BYTES);
336 #ifdef CONFIG_MODULES
337     if (size < PERCPU_ENOUGH_ROOM)
338         size = PERCPU_ENOUGH_ROOM;
339 #endif
340
341     ptr = alloc_bootmem(size * NR_CPUS);
342
343     for (i = 0; i < NR_CPUS; i++, ptr += size) {
344         __per_cpu_offset[i] = ptr - __per_cpu_start;
345         memcpy(ptr, __per_cpu_start, __per_cpu_end - __per_cpu_start);
346     }
347 }
-----
```

第 329~332 行: 初始化管理连续内存块的变量。在链接期间, “链接器魔数” 变量在相应体系结构的内核目录下定义 (例如, arch/i386/kernel/vmlinux.lds.s)。

第 334~341 行: 确定单个 CPU 所需内存的大小, 并为系统中每个 CPU 分配这样的内存, 就像一整块连续的内存一样。

第 343~346 行: 在新分配的内存间循环, 初始化每个 CPU 的内存块。在概念上, 我们取出了对单个 CPU 有效的一块数据 (从 __per_cpu_start 到 __per_cpu_end), 并将它复制给系统

中的每个 CPU，通过这种方式，每个 CPU 都可以操作自己的数据。

8.5.6 调用 `smp_prepare_boot_cpu()`

第 415 行：与 `smp_per_cpu_areas()` 类似，当 Linux 内核不支持 SMP 时，函数 `smp_prepare_boot_cpu()` 什么都不做：

```
-----
include/linux/smp.h
106 #define smp_prepare_boot_cpu()    do {} while (0)
-----
```

然而，如果 Linux 内核编译时支持 SMP，那么必须允许引导的 CPU 访问其控制台驱动程序，以及刚初始化过的每个 CPU 的存储器。可以通过设置 CPU 的位掩码来实现这一功能。

CPU 的位掩码定义如下：

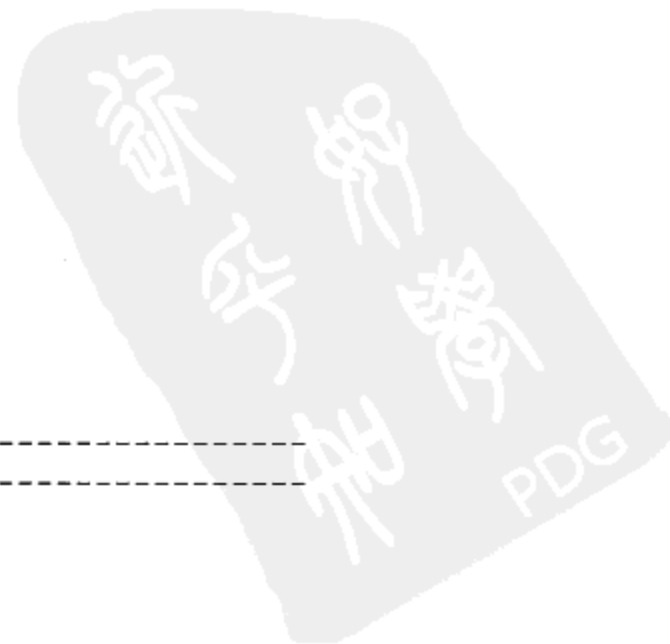
```
-----
include/asm-generic/cpumask.h
10 #if NR_CPUS > BITS_PER_LONG && NR_CPUS != 1
11 #define CPU_ARRAY_SIZE    BITS_TO_LONGS(NR_CPUS)
12
13 struct cpumask
14 {
15     unsigned long mask[CPU_ARRAY_SIZE];
16 };
-----
```

这就意味着与平台无关的位掩码包含与系统中 CPU 个数相同的位数。

`smp_prepare_boot_cpu()` 在 Linux 内核的与体系结构相关的部分实现，读者很快就会发现，i386 系统中和 PPC 系统中这部分代码完全一样：

```
-----
arch/i386/kernel/smpboot.c
66 /* bitmap of online cpus */
67 cpumask_t cpu_online_map;
...
70 cpumask_t cpu_callout_map;
...
1341 void __devinit smp_prepare_boot_cpu(void)
1342 {
1343     cpu_set(smp_processor_id(), cpu_online_map);
1344     cpu_set(smp_processor_id(), cpu_callout_map);
1345 }
-----
```

```
-----
arch/ppc/kernel/smp.c
49 cpumask_t cpu_online_map;
50 cpumask_t cpu_possible_map;
...
331 void __devinit smp_prepare_boot_cpu(void)
332 {
-----
```



```

333  cpu_set(smp_processor_id(), cpu_online_map);
334  cpu_set(smp_processor_id(), cpu_possible_map);
335  }

```

在这两个函数中, `cpu_set()` 仅在类型为 `cpumask_t` 的位图中设置 `smp_processor_id()` 位。这就意味着将该位的值置为 1。

8.5.7 调用 `sched_init()`

第 422 行: 在系统运行的过程中, 调用 `sched_init()` 标志着所有对象的初始化, 调度程序通过操作这些对象来管理 CPU 时间的分配。记住, 此时仅存在一个进程, 就是 `init` 进程, 该进程正在执行 `sched_init()`:

```

-----
kernel/sched.c
3896 void __init sched_init(void)
3897 {
3898     runqueue_t *rq;
3899     int i, j, k;
3900
3901     ...
3919     for (i = 0; i < NR_CPUS; i++) {
3920         prio_array_t *array;
3921
3922         rq = cpu_rq(i);
3923         spin_lock_init(&rq->lock);
3924         rq->active = rq->arrays;
3925         rq->expired = rq->arrays + 1;
3926         rq->best_expired_prio = MAX_PRIO;
3927
3928         ...
3938         for (j = 0; j < 2; j++) {
3939             array = rq->arrays + j;
3940             for (k = 0; k < MAX_PRIO; k++) {
3941                 INIT_LIST_HEAD(array->queue + k);
3942                 __clear_bit(k, array->bitmap);
3943             }
3944             // delimiter for bitsearch
3945             __set_bit(MAX_PRIO, array->bitmap);
3946         }
3947     }
3948     /*
3949     * We have to do a little magic to get the first
3950     * thread right in SMP mode.
3951     */
3952     rq = this_rq();
3953     rq->curr = current;
3954     rq->idle = current;
3955     set_task_cpu(current, smp_processor_id());
3956     wake_up_forked_process(current);
3957
3958     /*

```

```

3959  * The boot idle thread does lazy MMU switching as well:
3960  */
3961  atomic_inc(&init_mm.mm_count);
3962  enter_lazy_tlb(&init_mm, current);
3963  }

```

第 3919~3926 行：初始化每个 CPU 的运行队列：活动队列、过期队列、以及自旋锁都在此时被初始化。回顾第 7 章，函数 `spin_lock_init()` 将自旋锁设置为 1，表示数据对象已经被解锁。

图 8-13 描述了已初始化的运行队列。

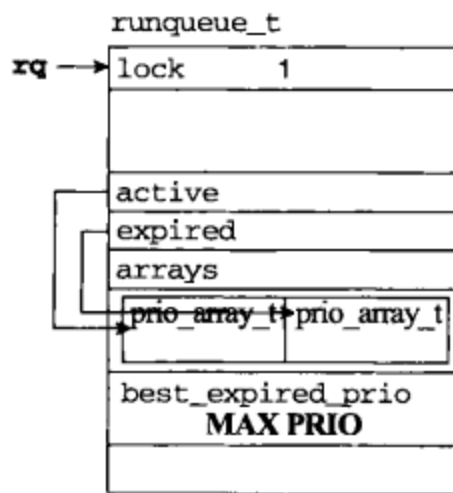


图 8-13 已初始化的运行队列 rq

第 3938~3947 行：对每个可能的优先级而言，初始化与优先级相关的链表并清除位图中所有的位，表示该队列中没有任何进程。（如果读者觉得此处不好理解的话，可以查阅图 8-14。另外，查阅第 7 章，可回顾调度程序是如何管理其运行队列的。）这段代码只是确保引入进程的一切工作都准备就绪了。从第 3947 行开始，调度程序已经就位，并知晓不存在任何进程，此时，它忽略了当前进程和空闲进程。

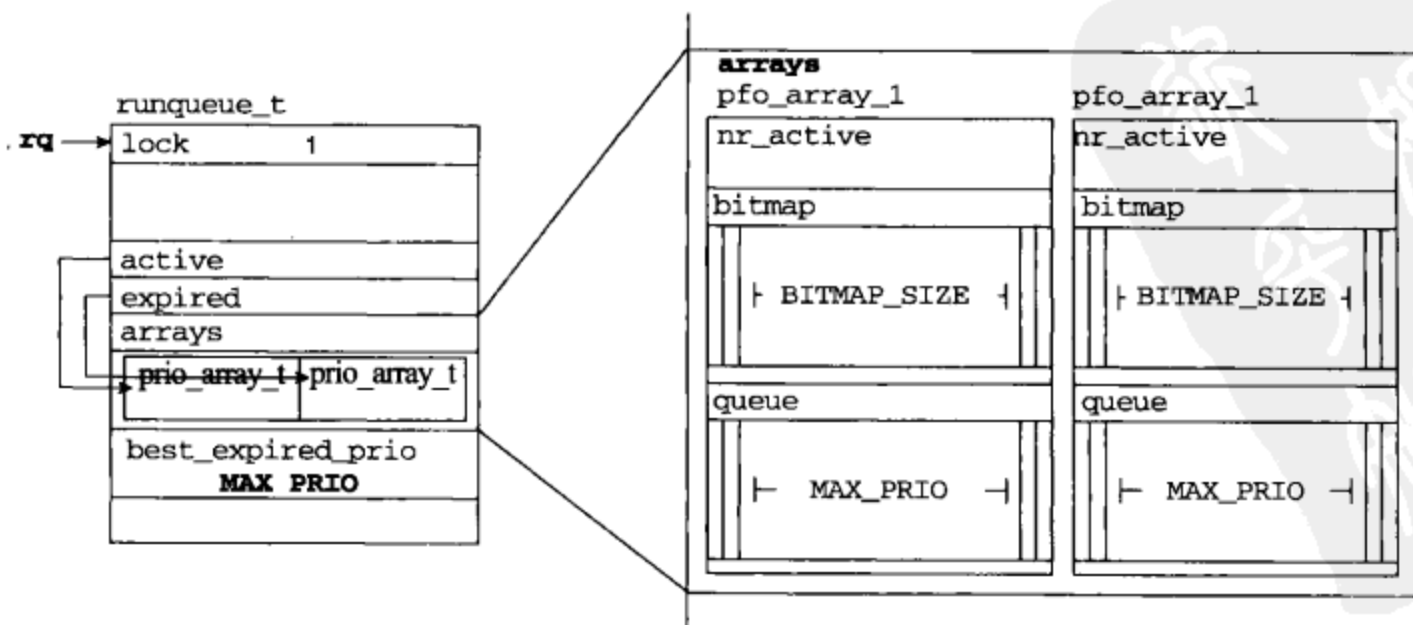


图 8-14 `rq->arrays`

第 3952~3956 行: 将当前进程加入当前 CPU 的运行队列, 并调用函数 `wake_up_forked_process()`, 以便初始化当前进程并通知调度程序。现在, 调度程序知道确实有一个进程了, 即 `init` 进程。

第 3961~3962 行: 当启用 MMU 交换技术时, 它允许多处理器 Linux 系统以更快的速度执行上下文切换。作为转换后备缓冲区, TLB 包含最近的页面转换地址。MMU 交换技术花费很长时间来刷新 TLB, 因此, 如果可能的话就交换 TLB 中的内容。函数 `enter_lazy_tlb()` 确保多 CPU 系统中没有 CPU 正在使用 `mm_struct init_mm`, 因而可以从容地进行交换。在单处理器系统中, 这是一个空函数。

上述代码忽略了用于处理 SMP 初始化的部分。概括起来, 那部分代码用于引导每个 CPU 获得默认的设置, 这些设置也是支持负载平衡、群组调度和线程迁移功能所必需的。为了简单起见, 此处忽略了这部分内容。

8.5.8 调用 `build_all_zonelists()`

第 424 行: 函数 `build_all_zonelists()` 根据页面区类型 `ZONE_DMA`、`ZONE_NORMAL` 和 `ZONE_HIGHMEM` 来划分内存空间。在第 6 章中曾提到, 页面区是物理存储器的线性划分, 主要用于突破硬件的限制, 只要知道这些页面区就是在该函数中创建的就足够了。页面区创建后, 页面就存储到属于页面区的页框中。

调用函数 `build_all_zonelists()`, 引入 `numnodes` 和 `NODE_DATA`。全局变量 `numnodes` 用于保存物理内存的节点 (或分区) 数。

分区是根据 CPU 的访问时间而定的。注意, 页表在此处就已经完全创建好了:

```
-----
mm/page_alloc.c
1345 void __init build_all_zonelists(void)
1346 {
1347     int i;
1348
1349     for(i = 0 ; i < numnodes ; i++)
1350         build_zonelists(NODE_DATA(i));
1351     printk("Built %i zonelists\n", numnodes);
1352 }
-----
```

函数 `build_all_zonelists()` 为每个结点调用一次 `build_zonelists()`, 打印出所创建的页面区列表数后, 该函数就结束了。本书并不深入探讨关于节点的更多内容。在单 CPU 的例子中, 只要知道 `numnodes` 等于 1 并且每个节点都有这三种页面区类型就足够了。宏 `NODE_DATA` 则从节点描述符列表中返回该节点的描述符。

8.5.9 调用 `page_alloc_init`

第 425 行: 函数 `page_alloc_init()` 仅向通告程序链 (notifier chain)^① 注册一个函数。已注

^① 在第 2 章中已经介绍过通告程序链。

册的函数 `page_alloc_cpu_notify()` 是一个涉及动态 CPU 配置的页摘除函数 (page-draining function) ^①。

动态 CPU 配置是指在 Linux 系统运行期间注册和撤销 CPU，涉及的事件被看做是“热插拔 CPU”。虽然从技术上讲，CPU 根本不是在机器操作期间插入和移除的，但在某些系统中可以开启和关闭，例如 IBM 的 p-Series690。让我们来看看这个函数：

```
-----
mm/page_alloc.c
1787 #ifdef CONFIG_HOTPLUG_CPU
1788 static int page_alloc_cpu_notify(struct notifier_block *self,
1789     unsigned long action, void *hcpu)
1790 {
1791     int cpu = (unsigned long)hcpu;
1792     long *count;
1793
1794     if (action == CPU_DEAD) {
1795         ...
1796         count = &per_cpu(nr_pagecache_local, cpu);
1797         atomic_add(*count, &nr_pagecache);
1798         *count = 0;
1799         local_irq_disable();
1800         __drain_pages(cpu);
1801         local_irq_enable();
1802     }
1803     return NOTIFY_OK;
1804 }
1805 #endif /* CONFIG_HOTPLUG_CPU */
1806
1807 void __init page_alloc_init(void)
1808 {
1809     hotcpu_notifier(page_alloc_cpu_notify, 0);
1810 }
-----
```

第 1809 行：本行向通告程序链 `hotcpu_notifier` 注册例程 `page_alloc_cpu_notify()`。例程 `hotcpu_notifier()` 创建一个结构体 `notifier_block`，该结构体指向函数 `page_alloc_cpu_notify()`，且其优先级为 0，然后在通告程序链 `cpu_chain`（位于 `kernel/cpu.c`）中注册一个对象。

第 1788 行：正如在第 2 章中所阐述的那样，`page_alloc_cpu_notify()` 有相应的通告程序调用参数，这个特定于系统的指针指向一个表示 CPU 序号的整数。

第 1794~1802 行：若该 CPU 已经死亡，则应该释放其所占用的页面。当 CPU 被杀死时，其行为就被设置为 `CPU_DEAD`。（可参阅同一文件中的函数 `drain_pages()`。）

8.5.10 调用 `parse_args()`

第 427 行：函数 `parse_args()` 用于解析传递给 Linux 内核的参数。

例如，`nfsroot` 是为无磁盘的系统设置 NFS 根文件系统的内核参数。读者可以在文件

^① 页摘除指的是如果 CPU 将不再被使用，那么就删除正在被该 CPU 使用的那些页。

Documentation/kernel-parameters.txt 中找到内核参数的完整列表。

```
-----
kernel/params.c
116 int parse_args(const char *name,
117     char *args,
118     struct kernel_param *params,
119     unsigned num,
120     int (*unknown)(char *param, char *val))
121 {
122     char *param, *val;
123
124     DEBUGP("Parsing ARGS: %s\n", args);
125
126     while (*args) {
127         int ret;
128
129         args = next_arg(args, &param, &val);
130         ret = parse_one(param, val, params, num, unknown);
131         switch (ret) {
132             case -ENOENT:
133                 printk(KERN_ERR "%s: Unknown parameter '%s'\n",
134                     name, param);
135                 return ret;
136             case -ENOSPC:
137                 printk(KERN_ERR
138                     "%s: '%s' too large for parameter '%s'\n",
139                     name, val ?: "", param);
140                 return ret;
141             case 0:
142                 break;
143             default:
144                 printk(KERN_ERR
145                     "%s: '%s' invalid for parameter '%s'\n",
146                     name, val ?: "", param);
147                 return ret;
148         }
149     }
150
151     /* All parsed OK. */
152     return 0;
153 }
-----
```

第 116~125 行: 传递给函数 `parse_args()` 的参数如下:

- ❑ **name**。当内核试图解析其参数变量时, 如果发生任何错误则显示一个字符串。在标准操作中, 这就意味着显示一条错误信息 “Booting Kernel: Unknown parameter X”。
- ❑ **args**。内核参数以 `foo=bar`、`bar2 baz=fuz wix` 的形式列出。
- ❑ **params**。指向内核参数的数据结构, 该结构体保存特定内核的所有有效参数。由于编译内核的方式不同, 有些参数可能存在, 有些可能不存在。
- ❑ **num**。这是该特定内核中的参数数目, 而不是 `args` 中的参数数目。

□ **unknown**。如果内核参数是未经验证而指定的，则 **unknown** 指向一个要调用的函数。

第 126~153 行：分解字符串 **args**，并将 **param** 设置为指向第一个参数的指针，**val** 为第一个参数值（如果有参数的话，否则 **val** 可以为空），这一操作是通过函数 **next_args()** 来实现的（例如，第一次用值为 **foo=bar,bar2 baz=fuz wix** 的参数来调用 **next_args()**）。我们将 **param** 设置为 **foo**，而 **val** 为 **bar,bar2**。**bar2** 后面的空间用 **\0** 来覆盖，**args** 则指向 **baz** 的起始字符。

可以看到，指针 **param** 和 **val** 被传给了 **parse_one()**，该函数用于设置实参的数据结构：

```
-----
kernel/params.c
46 static int parse_one(char *param,
47     char *val,
48     struct kernel_param *params,
49     unsigned num_params,
50     int (*handle_unknown)(char *param, char *val))
51 {
52     unsigned int i;
53
54     /* Find parameter */
55     for (i = 0; i < num_params; i++) {
56         if (parameq(param, params[i].name)) {
57             DEBUGP("They are equal! Calling %p\n",
58                 params[i].set);
59             return params[i].set(val, &params[i]);
60         }
61     }
62
63     if (handle_unknown) {
64         DEBUGP("Unknown argument: calling %p\n", handle_unknown);
65         return handle_unknown(param, val);
66     }
67
68     DEBUGP("Unknown argument '%s'\n", param);
69     return -ENOENT;
70 }
-----
```

第 46~54 行：这些参数与函数 **parse_args()** 中描述的指向部分参数的 **param** 和 **val** 相同。

第 55~61 行：分解已定义的内核参数，看看是否有和 **param** 匹配的参数。如果找到一个匹配的参数，就可以使用 **val** 来调用相关的集合函数。因此，集合函数可以处理多个或 0 个参数。

第 62~66 行：如果没有找到内核参数，就通过 **parse_args()** 来调用函数 **handle_unknown()**。

args 中每个指定的参数-值组合都调用过函数 **parse_one()** 后，内核参数就设置完毕，为继续启动 Linux 内核做好了准备。

8.5.11 调用 **trap_init()**

第 431 行：在第 3 章中已介绍过异常和中断。函数 **trap_init()** 在 x86 体系结构中是针对中

断处理操作的。简而言之,该函数初始化 x86 硬件引用的表,表中每个元素都有一个函数用于处理与内核和用户相关的问题,例如无效指令或引用了当前不在内存的页面。虽然 PowerPC 也可能存在相同的问题,但它处理这些问题的方式稍有不同(同样,所有相关内容已经在第 3 章讨论过了)。

8.5.12 调用 rcu_init()

第 432 行:函数 rcu_init()初始化 Linux 2.6 内核的读-复制-更新(Read-Copy-Update, RCU)子系统。RCU 控制访问临界区的代码,与芯片速度相比,当获取锁的代价变得非常高时,RCU 强制系统中的互斥现象。本书并不讨论 Linux 中 RCU 的实现,分析代码时偶尔会提到调用 RCU 子系统,但并不考虑其具体细节。有关 Linux 中 RCU 子系统的更多信息,可参阅 Linux Scalability Effort 的主页 <http://lse.sourceforge.net/locking/rcupdate.html>;

```
-----
kernel/rcupate.c
297 void __init rcu_init(void)
298 {
299     rcu_cpu_notify(&rcu_nb, CPU_UP_PREPARE,
300         (void *) (long) smp_processor_id());
301     /* Register notifier for non-boot CPUs */
302     register_cpu_notifier(&rcu_nb);
303 }
-----
```

8.5.13 调用 init_IRQ()

第 433 行: arch/i386/kernel/i8259.c 中的函数 init_IRQ()用于初始化硬件中断控制器和中断向量表,如果是 x86 体系结构的话,还要初始化系统定时器。第 3 章详细讨论了 x86 和 PPC 的中断,并以实时时钟为例:

```
-----
arch/i386/kernel/i8259.c
410 void __init init_IRQ(void)
411 {
412     int i;
413     ...
422     for (i = 0; i < (NR_VECTORS - FIRST_EXTERNAL_VECTOR); i++) {
423         int vector = FIRST_EXTERNAL_VECTOR + i;
424         if (i >= NR_IRQS)
425             break;
426         ...
430         if (vector != SYSCALL_VECTOR)
431             set_intr_gate(vector, interrupt[i]);
432     }
433     ...
437     intr_init_hook();
438     ...
443     setup_timer();
444     ...
-----
```

```

449 if (boot_cpu_data.hard_math && !cpu_has_fpu)
450     setup_irq(FPU_IRQ, &fpu_irq);
451 }

```

第 422~432 行：初始化中断向量。这就将 x86 的（硬件）IRQ 与相应的中断处理程序关联起来了。

第 437 行：设置特定于机器的 IRQ，例如高级可编程中断控制器（APIC）。

第 443 行：初始化定时器时钟。

第 449~450 行：需要的话，为 FPU 设置中断请求。

以下是函数 `init_IRQ()` 在 PPC 中的实现：

```

-----
arch/ppc/kernel/irq.c
700 void __init init_IRQ(void)
701 {
702     int i;
703
704     for (i = 0; i < NR_IRQS; ++i)
705         irq_affinity[i] = DEFAULT_CPU_AFFINITY;
706
707     ppc_md.init_IRQ();
708 }
-----

```

第 704 行：在多重处理器系统中，中断对特定的处理器有吸引力。

第 707 行：在 PowerMac 平台上，该例程位于文件 `arch/ppc/platforms/pmac_pic.c` 中，用于设置 I/O 控制器的可编程中断控制器（PIC）部分。

8.5.14 调用 `softirq_init()`

第 436 行：函数 `softirq_init()` 为作为引导的 CPU 接受来自 tasklet 的通知做好准备。接下来看看该函数的内部结构：

```

-----
kernel/softirq.c
317 void __init softirq_init(void)
318 {
319     open_softirq(TASKLET_SOFTIRQ, tasklet_action, NULL);
320     open_softirq(HI_SOFTIRQ, tasklet_hi_action, NULL);
321 }
...
327 void __init softirq_init(void)
328 {
329     open_softirq(TASKLET_SOFTIRQ, tasklet_action, NULL);
330     open_softirq(HI_SOFTIRQ, tasklet_hi_action, NULL);
331     tasklet_cpu_notify(&tasklet_nb, (unsigned long)CPU_UP_PREPARE,
332                      (void *) (long)smp_processor_id());
333     register_cpu_notifier(&tasklet_nb);
334 }
-----

```

第 319~320 行: 当获得 TASKLET_SOFTIRQ 或 HI_SOFTIRQ 中断时, 初始化要执行的操作。当传入一个 NULL 时, 就是告诉 Linux 内核去调用函数 tasklet_action(NULL) 和 tasklet_hi_action(NULL) (分别是第 319 行和第 320 行所列的情况)。以下是函数 open_softirq() 的实现, 它说明了 Linux 内核如何存放 tasklet 的初始化信息。

```
-----
kernel/softirq.c
177 void open_softirq(int nr, void (*action)(struct softirq_action*),
void * data)
178 {
179     softirq_vec[nr].data = data;
180     softirq_vec[nr].action = action;
181 }
-----
```

8.5.15 调用 time_init()

第 437 行: 函数 time_init() 选择并初始化系统定时器。与 trap_init() 类似, 该函数非常依赖于体系结构, 第 3 章探讨定时器中断时对此进行了讨论。系统定时器为 Linux 提供了一个临时视图, 当任务应当运行时, 允许它调度该任务并确定运行多长时间。Intel 的 HPET (High Performance Event Timer, 高性能事件定时器) 将逐渐取代 8254 PIT 和 RTC。HPET 采用内存映射 I/O 的方式, 这就意味着访问 HPET 控制寄存器就像访问内存地址一样。访问 I/O 区域时必须配置恰当的存储器。如果 arch/i386/defconfig.h 中有相应设置, 就得推迟函数 time_init() 的执行, 直到函数 mem_init() 设置完内存区。请参阅以下代码:

```
-----
arch/i386/kernel/time.c
376 void __init time_init(void)
377 {
...
378 #ifdef CONFIG_HPET_TIMER
379     if (is_hpet_capable()) {
380         late_time_init = hpet_time_init;
381         return;
382     }
...
387 #endif
388     xtime.tv_sec = get_cmos_time();
389     wall_to_monotonic.tv_sec = -xtime.tv_sec;
390     xtime.tv_nsec = (INITIAL_JIFFIES % HZ) * (NSEC_PER_SEC / HZ);
391     wall_to_monotonic.tv_nsec = -xtime.tv_nsec;
392
393     cur_timer = select_timer();
394     printk(KERN_INFO "Using %s for high-res timesource\n", cur_timer->name);
395
396     time_init_hook();
397 }
-----
```

第 379~387 行：如果配置了 HPET，函数 `time_init()` 就必须在初始化内存之后才能运行。函数 `late_time_init()` 的代码（见第 358~373 行）与函数 `time_init()` 相同。

第 388~391 行：初始化 `xtime` 时间结构体，用于保存时刻。

第 393 行：选择初始化的第一个定时器。这一步可以跳过。（详情请参见 `arch/i386/kernel/timers/timer.c`。）

8.5.16 调用 `console_init()`

第 444 行：计算机控制台是内核（以及系统中其他部分）输出信息的设备，同时也有注册（`login`）功能。根据系统的不同，控制台可以位于监视器上或经由串口实现。函数 `console_init()` 是初始化控制台设备的早期函数调用，它被允许在引导时报告状态信息：

```
-----
drivers/char/tty_io.c
2347 void __init console_init(void)
2348 {
2349     initcall_t *call;
2350     ...
2352     (void) tty_register_ldisc(N_TTY, &tty_ldisc_N_TTY);
2353     ...
2358 #ifdef CONFIG_EARLY_PRINTK
2359     disable_early_printk();
2360 #endif
2361     ...
2366     call = &__con_initcall_start;
2367     while (call < &__con_initcall_end) {
2368         (*call)();
2369         call++;
2370     }
2371 }
-----
```

第 2352 行：设置线路规程。

第 2359 行：需要时保留对早期 `printk` 的支持。完全初始化系统控制台之前，支持早期 `printk` 将允许系统在引导过程中报告状态信息，至少，它还能逐一初始化串口（如 `ttyS0`）和系统的 VGA。对早期 `printk` 的支持始于函数 `setup_arch()`。（详情可参阅本节对第 408 行代码的讨论和文件 `/kernel/printk.c`，以及 `/arch/i386/kernel/early_printk.c`。）

第 2366 行：初始化控制台。

8.5.17 调用 `profile_init()`

第 447 行：函数 `profile_init()` 为内核分配内存以存储分析数据（`profiling data`）。“分析”是一个术语，在计算机科学中用于描述在程序运行期间收集的数据。分析数据常用于分析程序的性能，或研究正在运行的程序（此处是指 Linux 内核本身）：

```
-----
kernel/profile.c
30 void __init profile_init(void)
```

```

31 {
32     unsigned int size;
33
34     if (!prof_on)
35         return;
36
37     /* only text is profiled */
38     prof_len = _etext - _stext;
39     prof_len >>= prof_shift;
40
41     size = prof_len * sizeof(unsigned int) + PAGE_SIZE - 1;
42     prof_buffer = (unsigned int *) alloc_bootmem(size);
43 }

```

第 34~35 行: 如果没有开启内核分析功能, 则什么都不做。

第 38~39 行: `_etext` 和 `_stext` 在 `kernel/head.S` 中定义, 根据它们的值来确定所要存放的数据的长度, 然后通过定义为内核参数的 `prof_shift` 来改变这个数据长度。

第 41~42 行: 为存储分析数据分配连续的内存块, 其大小由内核参数来确定。

8.5.18 调用 `local_irq_enable()`

第 448 行: `local_irq_enable()` 允许中断当前 CPU, 它常与 `local_irq_disable()` 成对使用。在早先版本的内核中, 函数 `sti()` 和 `cli()` 也有相同的功能。虽然这些宏仍然需要转化为函数 `sti()` 和 `cli()`, 但此处要注意的是关键词 `local`, 这些宏仅作用于当前正在运行的处理器。

```

-----
include\asm-i386\system.h

446 #define local_irq_disable() __asm__ __volatile__ ("cli": : : "memory")
447 #define local_irq_enable() __asm__ __volatile__ ("sti": : : "memory")
-----

```

第 446~447 行: 查阅 2.4 节, 引号中的语句就是汇编指令, 且内存位于已修改列表中。

8.5.19 配置 `initrd`

第 449~456 行: 这条 `#ifdef` 语句是对 `initrd` (即原始的 RAM 盘) 进行完全的检查。

系统使用 `initrd` 来加载内核, 并挂载原始的 RAM 盘作为根文件系统。程序可以从这个 RAM 盘上运行, 当时机成熟时, 就可以挂载新的根文件系统, 例如硬盘驱动器上的某个文件系统, 并卸载原始的 RAM 盘。

该操作仅能确保指定的原始 RAM 盘有效, 否则将 `initrd_start` 设置为 0, 以通知内核不要使用原始的 RAM 盘^①。

8.5.20 调用 `mem_init()`

第 457 行: 对 x86 和 PPC 而言, `mem_init()` 的功能都是找到所有空闲页面并发送该信息到

^① 详情可查阅 `Documentation/initrd.txt`。

控制台。第4章曾提到 Linux 内核将可用内存分为多个页面区。现在, Linux 有3个页面区。

- **zone_DMA**。16 M 以内的内存。
- **zone_Normal**。从 16 M 开始但小于 896 M 的内存。(内核使用最后的 128 M)
- **zone_HIGHMEM**。大于 1 G 的内存。

函数 `mem_init()` 找到所有页面区中空闲页面的总数, 并打印出关于内存初始状态的内核信息。该函数管理早期的内存分配数据, 因此它和体系结构相关。虽然是执行同一个任务, 但每种体系结构都提供了自己的函数来实现这一功能。接下来, 首先看看 x86 是如何实现这一函数的, 之后再讨论该函数在 PPC 中的实现:

```
-----
arch/i386/mm/init
445 void __init mem_init(void)
446 {
447     extern int ppro_with_ram_bug(void);
448     int codesize, reservedpages, datasize, initsize;
449     int tmp;
450     int bad_ppro;
451     ...
459 #ifdef CONFIG_HIGHMEM
460     if (PKMAP_BASE+LAST_PKMAP*PAGE_SIZE >= FIXADDR_START) {
461         printk(KERN_ERR "fixmap and kmap areas overlap - this will crash\n");
462         printk(KERN_ERR "pkstart: %lxh pkend: %lxh fixstart %lxh\n",
463             PKMAP_BASE, PKMAP_BASE+LAST_PKMAP*PAGE_SIZE, FIXADDR_START);
464         BUG();
465     }
466 #endif
467
468     set_max_mapnr_init();
469     ...
476 /* this will put all low memory onto the freelists */
477     totalram_pages += __free_all_bootmem();
478
479     reservedpages = 0;
480     for (tmp = 0; tmp < max_low_pfn; tmp++)
481         ...
485     if (page_is_ram(tmp) && PageReserved(pfn_to_page(tmp)))
486         reservedpages++;
487
488     set_highmem_pages_init(bad_ppro);
489     codesize = (unsigned long) &_amp;etext - (unsigned long) &_amp;text;
490     datasize = (unsigned long) &_amp;edata - (unsigned long) &_amp;etext;
491     initsize = (unsigned long) &_amp;__init_end - (unsigned long) &_amp;__init_begin;
492
493     kclist_add(&kcore_mem, __va(0), max_low_pfn << PAGE_SHIFT);
494     kclist_add(&kcore_vmalloc, (void *)VMALLOC_START,
495         VMALLOC_END-VMALLOC_START);
496
497     printk(KERN_INFO "Memory: %luk/%luk available (%dk kernel code, %dk
498         reserved, %dk data, %dk init, %ldk highmem)\n",
499         (unsigned long) nr_free_pages() << (PAGE_SHIFT-10),
```

```

500     num_physpages << (PAGE_SHIFT-10),
501     codesize >> 10,
502     reservedpages << (PAGE_SHIFT-10),
503     datasize >> 10,
504     initsize >> 10,
505     (unsigned long) (totalhigh_pages << (PAGE_SHIFT-10))
506     );
...
521 #ifndef CONFIG_SMP
522     zap_low_mappings();
523 #endif
524 }

```

第 459 行: 本行实现的是简单的错误检查功能, 从而使固定映像不会和内核映像重叠。

第 469 行: 函数 `set_max_mapnr_init()` (详情请参阅 `arch/i386/mm/init.c`) 仅设置 `num_physpages` 的值, 这是一个用于保存可用页面数的全局变量 (在 `mm/memory.c` 中定义)。

第 477 行: `__free_all_bootmem()` 用于标记所有正在释放的低端内存页面。引导时, 所有页面都被保留。此处, 将在引导阶段的稍后时刻释放可用的低端内存页面。该函数的调用流程参见图 8-15。

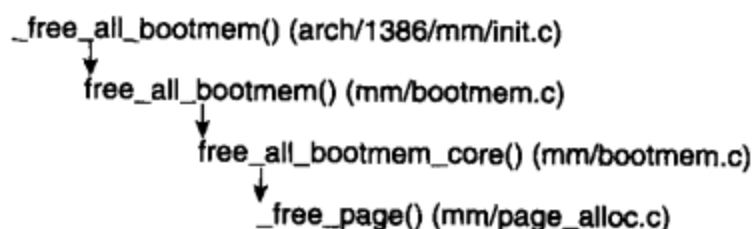


图 8-15 `__free_all_bootmem()` 调用的层次

让我们来看看 `free_all_bootmem_core()` 的核心部分, 以便理解它究竟是如何工作的:

```

-----
mm/bootmem.c
257 static unsigned long __init free_all_bootmem_core(pg_data_t *pgdat)
258 {
259     struct page *page;
260     bootmem_data_t *bdata = pgdat->bdata;
261     unsigned long i, count, total = 0;
...
295     page = virt_to_page(bdata->node_bootmem_map);
296     count = 0;
297     for (i = 0; i < ((bdata->node_low_pfn - (bdata->node_boot_start >>
PAGE_SHIFT)) / 8 + PAGE_SIZE - 1) / PAGE_SIZE; i++, page++) {
298         count++;
299         ClearPageReserved(page);
300         set_page_count(page, 1);
301         __free_page(page);
302     }
303     total += count;
304     bdata->node_bootmem_map = NULL;
305
306     return total;

```

```
307 }
```

对所有可用的低端内存页面而言,需要清除页结构中 flags 字段的 PG_reserved 标志^①。接下来,将页结构的 count 字段设置为 1,表示该页正在使用,然后调用 __free_page(),以便将该页传给伙伴分配程序。若你已回想起第 4 章阐述过的伙伴系统,就可以解释该函数如何释放一个页面并将其加入到空闲链表了。

函数 __free_all_bootmem() 返回低端内存的可用页面数,该值被加入 totalram_pages (在 mm/page_alloc.c 中定义的无符号长整型数) 的运行计数中。

第 480~486 行: 这些行更新预留页面的计数。

第 488 行: 调用 set_highmem_pages_init(), 标记高端内存页面的初始化。图 8-16 解释了该函数的调用层次。

```
set_highmem_pages_init()(arch/i386/mm/init.c)
    ↓
one_highpage_init()(arch/i386/mm/init.c)
```

图 8-16 highmem_pages_init 的调用层次

让我们来看看 one_highpage_init() 中执行的大部分代码:

```
arch/i386/mm/init.c
253 void __init one_highpage_init(struct page *page, int pfn, int bad_ppro)
254 {
255     if (page_is_ram(pfn) && !(bad_ppro && page_kills_ppro(pfn))) {
256         ClearPageReserved(page);
257         set_bit(PG_highmem, &page->flags);
258         set_page_count(page, 1);
259         __free_page(page);
260         totalhigh_pages++;
261     } else
262         SetPageReserved(page);
263 }
```

与 __free_all_bootmem() 非常相似,所有高端内存页面都有自己的由 PG_reserved 标志清除的页结构 flags 字段,也有 PG_highmem 集合,并将其 count 字段设置为 1。同时也可以调用 __free_page(), 将这些页面加入到空闲链表中,并将 totalhigh_pages 计数器加 1。

第 490~506 行: 该代码块收集并打印与存储区大小相关的信息以及可用页面数。

第 521~523 行: 函数 zap_low_mappings 刷新低端内存中初始化的 TLB 和 PGD。

函数 mem_init() 标记内存分配引导阶段的末尾,以及将贯穿整个系统的内存分配的开始。

PPC 中 mem_init() 的代码用于找到并初始化所有页面区的每个页面:

```
arch/ppc/mm/init.c
393 void __init mem_init(void)
```

① 回顾第 6 章,曾讨论过对已经在内存的页设置该标志。在早期的引导过程中,对低端内存也设置该标志。

```

394 {
395     unsigned long addr;
396     int codepages = 0;
397     int datapages = 0;
398     int initpages = 0;
399     #ifdef CONFIG_HIGHMEM
400     unsigned long highmem_mapnr;

402     highmem_mapnr = total_lowmem >> PAGE_SHIFT;
403     highmem_start_page = mem_map + highmem_mapnr;
404 #endif /* CONFIG_HIGHMEM */
405     max_mapnr = total_memory >> PAGE_SHIFT;

407     high_memory = (void *) __va(PPC_MEMSTART + total_lowmem);
408     num_physpages = max_mapnr; /* RAM is assumed contiguous */

410     totalram_pages += free_all_bootmem();

412 #ifdef CONFIG_BLK_DEV_INITRD
413     /* if we are booted from BootX with an initial ramdisk,
414        make sure the ramdisk pages aren't reserved. */
415     if (initrd_start) {
416         for (addr = initrd_start; addr < initrd_end; addr += PAGE_SIZE)
417             ClearPageReserved(virt_to_page(addr));
418     }
419 #endif /* CONFIG_BLK_DEV_INITRD */

421 #ifdef CONFIG_PPC_OF
422     /* mark the RTAS pages as reserved */
423     if (rtas_data)
424         for (addr = (ulong)__va(rtas_data);
425              addr < PAGE_ALIGN((ulong)__va(rtas_data)+rtas_size) ;
426              addr += PAGE_SIZE)
427             SetPageReserved(virt_to_page(addr));
428 #endif
429 #ifdef CONFIG_PPC_PMAC
430     if (agp_special_page)
431         SetPageReserved(virt_to_page(agp_special_page));
432 #endif
433     if (sysmap)
434         for (addr = (unsigned long)sysmap;
435              addr < PAGE_ALIGN((unsigned long)sysmap+sysmap_size) ;
436              addr += PAGE_SIZE)
437             SetPageReserved(virt_to_page(addr));

439     for (addr = PAGE_OFFSET; addr < (unsigned long)high_memory;
440          addr += PAGE_SIZE) {
441         if (!PageReserved(virt_to_page(addr)))
442             continue;
443         if (addr < (ulong) etext)

```

```
444     codepages++;
445     else if (addr >= (unsigned long)&__init_begin
446             && addr < (unsigned long)&__init_end)
447         initpages++;
448     else if (addr < (ulong) klimit)
449         datapages++;
450 }

452 #ifdef CONFIG_HIGHMEM
453 {
454     unsigned long pfn;

456     for (pfn = highmem_mapnr; pfn < max_mapnr; ++pfn) {
457         struct page *page = mem_map + pfn;

459         ClearPageReserved(page);
460         set_bit(PG_highmem, &page->flags);
461         set_page_count(page, 1);
462         __free_page(page);
463         totalhigh_pages++;
464     }
465     totalram_pages += totalhigh_pages;
466 }
467 #endif /* CONFIG_HIGHMEM */

469 printk("Memory: %luk available (%dk kernel code, %dk data, %dk init, %ldk
highmem)\n",
470        (unsigned long)nr_free_pages() << (PAGE_SHIFT-10),
471        codepages << (PAGE_SHIFT-10), datapages << (PAGE_SHIFT-10),
472        initpages << (PAGE_SHIFT-10),
473        (unsigned long) (totalhigh_pages << (PAGE_SHIFT-10)));
474 if (sysmap)
475     printk("System.map loaded at 0x%08x for debugger, size: %ld bytes\n",
476            (unsigned int)sysmap, sysmap_size);
477 #ifdef CONFIG_PPC_PMAC
478     if (agp_special_page)
479         printk(KERN_INFO "AGP special page: 0x%08lx\n", agp_special_page);
480 #endif

482 /* Make sure all our pagetable pages have page->mapping
483    and page->index set correctly. */
484 for (addr = KERNELBASE; addr != 0; addr += PGDIR_SIZE) {
485     struct page *pg;
486     pmd_t *pmd = pmd_offset(pgd_offset_k(addr), addr);
487     if (pmd_present(*pmd)) {
488         pg = pmd_page(*pmd);
489         pg->mapping = (void *) &init_mm;
490         pg->index = addr;
491     }
492 }

493 mem_init_done = 1;
494 }
```

第 399~410 行: 这些行找出可用内存的总数。如果使用了 HIGHMEM, 那么高端内存页面也要计入总数, 可以通过修改全局变量 `totalram_pages` 来反映这一点。

第 412~419 行: 如果使用了原始的 RAM 盘, 就清除引导 RAM 盘使用过的任何页面。

第 421~432 行: 如果需要的话, 可根据引导环境的不同, 为实时抽象服务和 AGP (视频) 预留所需的页面。

第 433~450 行: 需要时为系统映像预留页面。

第 452~467 行: 如果正在使用 HIGHMEM, 那么清除任何预留的页面, 并修改全局变量 `totalram_pages`。

第 469~480 行: 将内存信息输出到控制台。

第 482~492 行: 重新回到页目录并初始化每个 `mm_struct` 结构和索引。

8.5.21 调用 `late_time_init()`

第 459~460 行: 函数 `late_time_init()` 使用 HPET (详情可参见 8.5.15 节)。该函数仅用于 Intel 体系结构和 HPET, 它本质上与 `time_init()` 相同, 只不过是在内存初始化后才调用, 以允许将 HPET 映射到物理内存。

8.5.22 调用 `calibrate_delay()`

第 461 行: `init/main.c` 中的函数 `calibrate_delay()` 用于计算并打印非常著名的“BogoMips”的值, 该值用于度量处理器在一个时钟节拍内可以反复执行多少个 `delay()`。对不同速度的处理器而言, `calibrate_delay()` 允许的延迟大致相同。处理器最多能运行多快, 其结果存储在 `loop_pre_jiffy` 中, 函数 `udelay()` 和 `mdelay()` 使用该值来设置反复执行 `delay()` 的次数:

```
-----
init/main.c
void __init calibrate_delay(void)
{
    unsigned long ticks, loopbit;
    int lps_precision = LPS_PREC;

186    loops_per_jiffy = (1<<12);

    printk("Calibrating delay loop... ");
189    while (loops_per_jiffy <= 1) {
        /* wait for "start of" clock tick */
        ticks = jiffies;
        while (ticks == jiffies)
            /* nothing */;
        /* Go .. */
        ticks = jiffies;
        __delay(loops_per_jiffy);
        ticks = jiffies - ticks;
        if (ticks)
            break;
200    }
```

```

/* Do a binary approximation to get loops_per_jiffy set to equal one clock
(up to lps_precision bits) */
204 loops_per_jiffy >>= 1;
    loopbit = loops_per_jiffy;
206 while ( lps_precision-- && (loopbit >>= 1) ) {
    loops_per_jiffy |= loopbit;
    ticks = jiffies;
    while (ticks == jiffies);
    ticks = jiffies;
    __delay(loops_per_jiffy);
    if (jiffies != ticks) /* longer than 1 tick */
        loops_per_jiffy &= ~loopbit;
214 }

/* Round the value and print it */
217 printk("%lu.%02lu BogoMIPS\n",
    loops_per_jiffy/(500000/HZ),
219 (loops_per_jiffy/(5000/HZ)) % 100);
}
-----

```

第 186 行：从地址 0x800 处开始。

第 189~200 行：将 loops_per_jiffy 加倍，直到它执行函数 delay(loops_per_jiffy) 所花费的总时间超过一个 jiffy。

第 204 行：loops_per_jiffy 除以 2。

第 206~214 行：将 loops_per_jiffy 连续加上 2 的递减次幂，直至节拍数等于 jiffy。

第 217~219 行：以浮点数的形式打印出该值。

8.5.23 调用 pgtable_cache_init()

第 463 行：x86 代码块中关键的函数是系统函数 kmem_cache_create()，该函数创建一个指定的缓冲区。它的第一个参数是一个字符串，常用于在 /proc/slabinfo 中识别该函数：

```

-----
arch/i386/mm/init.c
529 kmem_cache_t *pgd_cache;
530 kmem_cache_t *pmd_cache;
531
532 void __init pgtable_cache_init(void)
533 {
534     if (PTRS_PER_PMD > 1) {
535         pmd_cache = kmem_cache_create("pmd",
536             PTRS_PER_PMD*sizeof(pmd_t),
537             0, 538             SLAB_HWCACHE_ALIGN | SLAB_MUST_H  WCACHE_ALIGN,
539             pmd_ctor,
540             NULL);
541         if (!pmd_cache)
542             panic("pgtable_cache_init(): cannot create pmd c  ache");
543     }
544     pgd_cache = kmem_cache_create("pgd",
545         PTRS_PER_PGD*sizeof(pgd_t),

```



```

546         0,
547         SLAB_HWCACHE_ALIGN | SLAB_MUST_HWCACHE_ALIGN,
548         pgd_ctor,
549         PTRS_PER_PMD == 1 ? pgd_dtor : NULL);
550     if (!pgd_cache)
551         panic("pgtable_cache_init(): Cannot create pgd cache");
552 }
-----
arch/ppc64/mm/init.c
976 void pgtable_cache_init(void)
977 {
978     zero_cache = kmem_cache_create("zero",
979         PAGE_SIZE,
980         0,
981         SLAB_HWCACHE_ALIGN | SLAB_MUST_HWCACHE_ALIGN,
982         zero_ctor,
983         NULL);
984     if (!zero_cache)
985         panic("pgtable_cache_init(): could not create zero_cache !\n");
986 }
-----

```

第 532~542 行: 创建高速缓存 pmd。

第 544~551 行: 创建高速缓存 pgd。

PPC 中有硬件辅助散列法, 因而 pgtable_cache_init() 是一个空操作函数:

```

-----
include\asmppc/pgtable.h
685 #define pgtable_cache_init() do { } while (0)
-----

```

8.5.24 调用 buffer_init()

第 472 行: fs/buffer.c 中的函数 buffer_init() 用于保存从文件系统设备获得的数据:

```

-----
fs/buffer.c
3031 void __init buffer_init(void)
3032 {
3033     int i;
3034     int nrpages;

3036     bh_cachep = kmem_cache_create("buffer_head",
3037         sizeof(struct buffer_head), 0,
3038         0, init_buffer_head, NULL);
3039     for (i = 0; i < ARRAY_SIZE(bh_wait_queue_heads); i++)
3040         init_waitqueue_head(&bh_wait_queue_heads[i].wqh);

3044     nrpages = (nr_free_buffer_pages() * 10) / 100;
3045     max_buffer_heads = nrpages * (PAGE_SIZE / sizeof(struct buffer_head));
3046     hotcpu_notifier(buffer_cpu_notify, 0);
3048 }
-----

```

第 3036 行：分配缓冲区散列表。

第 3039 行：创建散列等待队列缓冲区表。

第 3044 行：将低端内存限制到 10%。

8.5.25 调用 `security_scaffolding_startup()`

第 474 行：Linux 2.6 内核包含加载内核模块的代码，这些内核模块可实现各种安全特性。`security_scaffolding_startup()` 仅检验一个安全操作对象是否存在，如果存在则调用安全模块的初始化函数。

本书并不讨论如何创建安全模块以及作者可能要面对什么样的问题，详情可查阅 Linux 安全模块 (<http://lsm.immunix.org/>) 和 Linux 安全模块邮件列表 (<http://mail.wirex.com/mailman/listinfo/linux-security-module>)。

8.5.26 调用 `vfs_caches_init()`

第 475 行：VFS 子系统依赖于内存缓存，也叫做 SLAB 缓存，来保存它所管理的数据结构。第 4 章已经详细讨论过 SLAB 缓存。函数 `vfs_caches_init()` 初始化子系统所使用的 SLAB 缓存，图 8-17 简单展示了它所调用的主要函数的层次结构。我们将详细讨论这些函数。具体分析每个函数时，参考该层次关系将有助于理解这些函数。

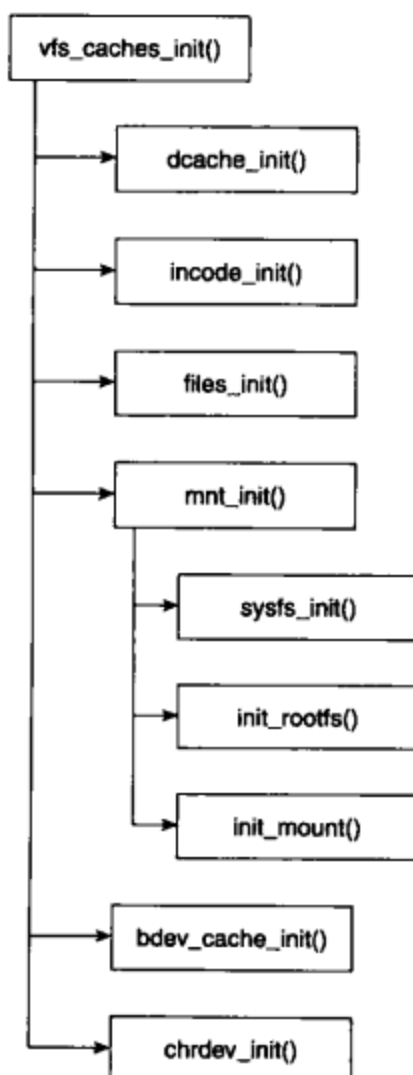


图 8-17 `vfs_caches_init()` 的调用层次

表 8-4 总结了由 `vfs_caches_init()` 和它所调用的某个函数引入的对象。

表8-4 `vfs_caches_init()`引入的对象

对 象 名	描 述
<code>names_cache</code>	全局变量
<code>filp_cache</code>	全局变量
<code>inode_cache</code>	全局变量
<code>dentry_cache</code>	全局变量
<code>mnt_cache</code>	全局变量
<code>namespace</code>	指向结构体的指针
<code>mount_hashtable</code>	全局变量
<code>root_fs_type</code>	全局变量
<code>file_system_type</code>	结构体 (参见第6章)
<code>bdev_cache</code>	全局变量

```

-----
fs/dcache.c
1623 void __init vfs_caches_init(unsigned long mempages)
1624 {
1625     names_cache = kmem_cache_create("names_cache",
1626         PATH_MAX, 0,
1627         SLAB_HWCACHE_ALIGN, NULL, NULL);
1628     if (!names_cache)
1629         panic("Cannot create names SLAB cache");
1630
1631     filp_cache = kmem_cache_create("filp",
1632         sizeof(struct file), 0,
1633         SLAB_HWCACHE_ALIGN, filp_ctor, filp_dtor);
1634     if (!filp_cache)
1635         panic("Cannot create filp SLAB cache");
1636
1637     dcache_init(mempages);
1638     inode_init(mempages);
1639     files_init(mempages);
1640     mnt_init(mempages);
1641     bdev_cache_init();
1642     chrdev_init();
1643 }
-----

```

第 1623 行: 该例程用全局变量 `num_physpages` (在 `mem_init()` 中计算该变量的值) 作为参数, 来保存在系统内存中可用的物理页面数。稍后我们将看到, 该参数值会影响 SLAB 缓存的创建。

第 1625~1629 行: 下一步就是创建内存区 `names_cache`。第 4 章详细描述了函数 `kmem_cache_create()`。该内存区保存大小为 `PATH_MAX` 的对象, `PATH_MAX` 则限定了路径名最大允许包含的字符数 (其数值在文件 `linux/limits.h` 中设置为 4096)。此处, 已创建的缓存中还没有对象, 或没有大小为 `PATH_MAX` 的内存区, 实际的内存区将在随后首次调用 `getname()` 时被分配。

第 6 章已经讨论过, 某些与文件相关的系统调用 (如 `sys_open()`) 一开始就调用了 `getname()`

例程，以便从进程的地址空间读取文件路径名，`putname()` 例程则从缓存释放对象。

如果无法创建缓存 `names_cache`，内核就会跳转到恐慌（panic）例程，从而退出该函数的控制流程。

第 1631~1635 行：下一步就是创建缓存 `filp_cache` 以及大小等于文件结构的对象。例程 `get_empty_filp()`（参见文件 `fs/file_table.c`）用于分配这个保存文件结构的对象，该例程将在创建管道或打开文件时被调用。调用例程 `file_free()`（参见文件 `fs/file_table.c`）则可回收该文件描述符对象。

第 1637 行：例程 `dcache_init()`（参见文件 `fs/dcache.c`）用于创建 SLAB 缓存，该缓存保存目录项描述符^①。缓存本身被称作 `dentry_cache`。当进程访问文件或目录时所涉及的路径名由多个路径分量组成，目录项描述符就是针对每个分量而创建的。目录项结构把文件或目录分量与其索引节点结合起来，因而通过该目录项可以更快地找到与其对应的索引节点。

第 1638 行：例程 `inode_init()`（参见文件 `fs/inode.c`）初始化索引节点散列表和由等待队列的队首节点构成的数组，该数组用于存储内核将要锁存的散列的索引节点。散列的索引节点的等待队列队首节点（`wait_queue_head_t`）存储在数组 `i_wait_queue_heads` 中，该数组将在系统的启动过程中被初始化。

此处创建 `inode_hashtable`，该表可以加速索引节点的查找。最后，SLAB 缓冲常用于保存创建的索引节点对象，称为 `inode_cache`。可以调用 `alloc_inode`（参见文件 `fs/inode.c`）来分配该缓存所在的内存区，而调用 `destroy_inode()`（参见文件 `fs/inode.c`）来释放这些内存区。

```
-----
fs/file_table.c
292 void __init files_init(unsigned long mempages)
293 {
294     int n;
...
299     n = (mempages * (PAGE_SIZE / 1024)) / 10;
300     files_stat.max_files = n;
301     if (files_stat.max_files < NR_FILE)
302         files_stat.max_files = NR_FILE;
303 }
```

第 299 行：用文件（包括其索引节点和缓存）大概占用的空间总量除以页面大小（这种情况下是 1 K），得到的值再乘以页数就得到了文件所占用的总块数。除以 10 表示其默认限制文件使用的内存不得超过可用内存的 10%。

第 301~302 行：将 `NR_FILE`（参见文件 `include/linux/fs.h`）设置为 8 192。

第 1640 行：下一个例程 `mnt_init()` 创建了用于保存 `vfsmount` 对象且名为 `mnt_cache` 的缓存，VFS 利用这些对象来挂载文件系统。该例程也创建 `mount_hashtable` 数组，该数组用于存放 `mnt_cache` 中对象引用，以便快速访问对象。然后，该例程发出调用来初始化 `sysfs` 文件系统并挂载 `root` 文件系统。接下来将考虑如何创建散列表：

① 回想一下，`dentry` 就是目录项 `directory entry` 的缩写。

```

-----
fs/namespace.c
1137 void __init mnt_init(unsigned long mempages)
{
1139     struct list_head *d;
1140     unsigned long order;
1141     unsigned int nr_hash;
1142     int i;
    ...
1149     order = 0;
1150     mount_hashtable = (struct list_head *)
1151         __get_free_pages(GFP_ATOMIC, order);
1152
1153     if (!mount_hashtable)
1154         panic("Failed to allocate mount hash table\n");
    ...
1161     nr_hash = (1UL << order) * PAGE_SIZE / sizeof(struct list_head);
1162     hash_bits = 0;
1163     do {
1164         hash_bits++;
1165     } while ((nr_hash >> hash_bits) != 0);
1166     hash_bits--;
    ...
1172     nr_hash = 1UL << hash_bits;
1173     hash_mask = nr_hash - 1;
1174
1175     printk("Mount-cache hash table entries: %d (order: %ld, %ld bytes)\n",
        nr_hash, order, (PAGE_SIZE << order));
    ...
1179     d = mount_hashtable;
1180     i = nr_hash;
1181     do {
1182         INIT_LIST_HEAD(d);
1183         d++;
1184         i--;
1185     } while (i);
    ..
1189 }
-----

```

第 1139~1144 行: 散列表队列由整个内存页面组成。在第 4 章中已经详细阐释了例程 `__get_free_pages()` 是如何工作的。简单地说, 该例程将返回指向大小为 2 的幂次个页面的内存区指针。因此, 我们分配一个页面来保存散列表。

第 1161~1173 行: 下一步就是确定表中的项数。`nr_hash` 保存散列表中链表头的次幂数 (2 的幂次)。`hash_bits` 是 `nr_hash` 中用于表示 2 的最高次幂所需的二进制位数。然后, 第 1172 行重新定义 `nr_hash` 而构成最左边的一位, 这样, 位掩码就可以根据新的 `nr_hash` 值来计算。

第 1179~1185 行: 最后, 调用宏 `INIT_LIST_HEAD` 来初始化散列表。该宏的参数是一个指针, 指向将要被初始化的新链表表头的内存区。这一过程将重复 `nr_hash` 次 (或该表可以容纳的项数)。

我们来看一个例子: 假定 `PAGE_SIZE` 是 4 KB, 而结构体 `list_head` 是 8 字节。因为 `order`

=0, nr_hash 的值变为 500; 也就是说, 在一个 4 KB 的表中可以填充 500 个 list_head 结构体。(1UL<<order) 变为已分配的页数。例如, 若 order 为 1 (表示已经要求为散列表分配 2 个页面), 0000 0001 左移一位变为 0000 0010 (在十进制计数法中是乘以 2)。接下来, 计算散列关键值所需的位数。预排该循环中的每次迭代, 可得到如下结果。

初始值: hash_bit=0, nr_hash=500。

- 第 1 次循环: hash_bit=1, (500>>1) !=0
(0001 1111 0100 >>1) = 0000 1111 1010
- 第 2 次循环: hash_bit=2, (500>>2) !=0
(0001 1111 1010 >>2) = 0000 0111 1110
- 第 3 次循环: hash_bit=3, (500>>3) !=0
(0001 1111 1010 >>3) = 0000 0011 1111
- 第 4 次循环: hash_bit=4, (500>>4) !=0
(0001 1111 1010 >>4) = 0000 0001 1111
- 第 5 次循环: hash_bit=5, (500>>5) !=0
(0001 1111 1010 >>5) = 0000 0000 1111
- 第 6 次循环: hash_bit=6, (500>>6) !=0
(0001 1111 1010 >>6) = 0000 0000 0111
- 第 7 次循环: hash_bit=7, (500>>7) !=0
(0001 1111 1010 >>7) = 0000 0000 0011
- 第 8 次循环: hash_bit=8, (500>>8) !=0
(0001 1111 1010 >>8) = 0000 0000 0001
- 第 9 次循环: hash_bit=9, (500>>9) !=0
(0001 1111 1010 >>9) = 0000 0000 0000

跳出 while 循环后, hash_bits 将减至 8, nr_hash 设为 0001 0000 0000, 而 hash_mask 被设为 0000 1111 1111。

例程 mnt_init() 在初始化 mount_hashtable 并创建 mnt_cache 后, 将发出如下 3 个调用:

```
-----
fs/namespace.c
...
1189  sysfs_init();
1190  init_rootfs();
1191  init_mount_tree();
1192 }
```

sysfs_init() 负责创建 sysfs 文件系统。init_rootfs() 和 init_mount_tree() 负责挂载根文件系统。下面将依次考虑每个例程的具体实现。

```
-----
init_rootfs()
fs/ramfs/inode.c
218  static struct file_system_type rootfs_fs_type = {
```

```

219     .name     = "rootfs",
220     .get_sb    = rootfs_get_sb,
221     .kill_sb   = kill_litter_super,
222 };
...
237 int __init init_rootfs(void)
238 {
239     return register_filesystem(&rootfs_fs_type);
240 }

```

rootfs 文件系统是内核最初挂载的文件系统，它是一个简单的空目录，稍后在内核引导的过程中挂载了真正的文件系统后，它就是多余的了。

第 218~222 行：这一代码块声明结构体 `rootfs_fs_type file_system_type`，此处只定义了这两种获得和摘除相关超级块的方法。

第 237~240 行：`init_rootfs()` 例程仅向内核注册这个 `rootfs`，这使得在内核中可以使用与文件系统类型相关的所有信息（这些信息被存储在结构体 `file_system_type` 中）。

```

-----
init_mount_tree()
fs/namespace.c
1107 static void __init init_mount_tree(void)
1108 {
1109     struct vfsmount *mnt;
1110     struct namespace *namespace;
1111     struct task_struct *g, *p;
1112
1113     mnt = do_kern_mount("rootfs", 0, "rootfs", NULL);
1114     if (IS_ERR(mnt))
1115         panic("Can't create rootfs");
1116     namespace = kmalloc(sizeof(*namespace), GFP_KERNEL);
1117     if (!namespace)
1118         panic("Can't allocate initial namespace");
1119     atomic_set(&namespace->count, 1);
1120     INIT_LIST_HEAD(&namespace->list);
1121     init_rwsem(&namespace->sem);
1122     list_add(&mnt->mnt_list, &namespace->list);
1123     namespace->root = mnt;
1124
1125     init_task.namespace = namespace;
1126     read_lock(&tasklist_lock);
1127     do_each_thread(g, p) {
1128         get_namespace(namespace);
1129         p->namespace = namespace;
1130     } while_each_thread(g, p);
1131     read_unlock(&tasklist_lock);
1132
1133     set_fs_pwd(current->fs, namespace->root,
1134               namespace->root->mnt_root);
1134     set_fs_root(current->fs, namespace->root,
1135               namespace->root->mnt_root);
1135 }
-----

```


第 1116~1123 行：初始化进程的命名空间。该结构保存指向与挂载树相关的结构及其相应目录项的指针。并分配 namespace 对象，将其计数值设置为 1，初始化 list_head 类型的 list 字段，初始化对命名空间（和挂载树）加锁的信号，将 vfsmount 结构对应的 root 字段设置为指向新分配的 vfsmount。

第 1125 行：将当前任务（即 init 任务）的进程描述符命名空间设置为指向刚分配并初始化的命名空间对象。（当前进程是进程 0。）

第 1134~1135 行：下列两个例程设置与该进程相关的 fs_struct 的四个字段的值。fs_struct 保存由这些例程设置的根目录和当前工作目录字段。

最后，分析 mnt_init 函数中究竟实现了哪些操作。现在，让我们继续来看看 vfs_mnt_init。

```
-----
1641 bdev_cache_init()
fs/block_dev.c
290 void __init bdev_cache_init(void)
291 {
292     int err;
293     bdev_cachep = kmem_cache_create("bdev_cache",
294         sizeof(struct bdev_inode),
295         0,
296         SLAB_HWCACHE_ALIGN|SLAB_RECLAIM_ACCOUNT,
297         init_once,
298         NULL);
299     if (!bdev_cachep)
300         panic("Cannot create bdev_cache SLAB cache");
301     err = register_filesystem(&bd_type);
302     if (err)
303         panic("Cannot register bdev pseudo-fs");
304     bd_mnt = kern_mount(&bd_type);
305     err = PTR_ERR(bd_mnt);
306     if (IS_ERR(bd_mnt))
307         panic("Cannot create bdev pseudo-fs");
308     blockdev_superblock = bd_mnt->mnt_sb; /* For writeback */
309 }
```

第 293~298 行：创建 SLAB 缓存 bdev_cache，该缓存用于保存 bdev_inodes。

第 301 行：注册特殊文件系统 bdev。该文件系统定义如下：

```
-----
fs/block_dev.c
294 static struct file_system_type bd_type = {
295     .name    = "bdev",
296     .get_sb  = bd_get_sb,
297     .kill_sb = kill_anon_super,
298 };
-----
```

如上所示，特殊文件系统 bdev 的 file_system_type 结构仅定义了两个例程：一个用于获取文件系统的超级块，另一个用于删除/释放超级块。读者可能会好奇为什么块设备要像文件系统一样向内核注册。在第 6 章中，我们已经看到，从技术上讲，那些并不是文件系统的系统也可

以使用文件系统的内核结构,也就是说,它们并没有挂载点,但可以使用所支持的文件系统的 VFS 内核结构。块设备就是这样一个使用 VFS 文件系统内核结构的伪文件系统。有了 bdev 后,这些特殊的文件系统通常仅需定义有限的字段,对某些特殊应用而言,并不是所有的字段都有意义。

第 304~308 行:调用 `kem_mount()` 来设置所有和挂载相关的 VFS 结构,并返回 `vfsmount` 结构。(有关如何设置全局变量 `bd_mnt` 使之指向 `vfsmount` 结构,以及设置 `blockdev_superblock`,使之指向 `vfsmount` 超级块,详情可参阅第 6 章)。

该函数用于初始化围绕驱动程序模型的字符设备对象:

```
-----
1642 chrdev_init
fs/char_dev.c
void __init chrdev_init(void)
{
433     subsystem_init(&cdev_subsys);
434     cdev_map = kobj_map_init(base_probe, &cdev_subsys);
435 }
```

8.5.27 调用 `radix_tree_init()`

第 476 行:Linux 2.6 内核使用基树 (radix tree) 来管理页高速缓存中的页面。此处,仅初始化内核空间毗邻的部分,以便存放页高速缓存的基树:

```
-----
lib/radix-tree.c
798 void __init radix_tree_init(void)
799 {
800     radix_tree_node_cachep = kmem_cache_create("radix_tree_node",
801         sizeof(struct radix_tree_node), 0,
802         SLAB_PANIC, radix_tree_node_ctor, NULL);
803     radix_tree_init_maxindex();
804     hotcpu_notifier(radix_tree_callback, 0);
-----

lib/radix-tree.c
768 static __init void radix_tree_init_maxindex(void)
769 {
770     unsigned int i;
771
772     for (i = 0; i < ARRAY_SIZE(height_to_maxindex); i++)
773         height_to_maxindex[i] = __maxindex(i);
774 }
```

请注意看看 `radix_tree_init()` 究竟是如何分配页高速缓存空间的,而 `radix_tree_init_maxindex()` 又是如何配置基树的数据存储 `height_to_maxindex[]` 的。

`hotcpu_notifier()` (见第 804 行) 是指 Linux 2.6 热交换 CPU 的性能。当 CPU 被热交换时,内核就调用 `radix_tree_callback()`, 该函数用于尝试释放连接到热交换 CPU 的页高速缓存。

8.5.28 调用 `signal_init()`

第 477 行: `kernel/signal.c` 中的函数 `signals_init()` 将初始化内核信号队列。

```
-----
fs/buffer.c
2565 void __init signals_init(void)
2566 {
2567     sigqueue_cachep =
2568         kmem_cache_create("sigqueue",
2569             sizeof(struct sigqueue),
2570             __alignof__(struct sigqueue),
2571             0, NULL, NULL);
2572     if (!sigqueue_cachep)
2573         panic("signals_init(): cannot create sigqueue SLAB cache");
2574 }
```

第 2567~2571 行: 为 `sigqueue` 分配 SLAB 内存。

8.5.29 调用 `page_writeback_init()`

第 479 行: 函数 `page_writeback_init()` 初始化一些值, 当脏页写回磁盘时这些值起控制作用。脏页并不立即写回磁盘, 而是当一定时间后或内存中一定数目和比例的页面被标记为脏页时才写回磁盘。该 `init` 函数尝试确定在触发后台回写和专门的回写操作之前脏页的最佳值。后台脏页回写比专门的脏页回写花费的处理器时间要少得多。

```
-----
mm/page-writeback.c
488 /*
489 * If the machine has a large highmem:lowmem ratio then scale back the
490 * default
491 * dirty memory thresholds: allowing too much dirty highmem pins an excessive
492 * number of buffer_heads.
493 */
494 void __init page_writeback_init(void)
495 {
496     long buffer_pages = nr_free_buffer_pages();
497     long correction;
498     total_pages = nr_free_pagecache_pages();
499     correction = (100 * 4 * buffer_pages) / total_pages;
500     if (correction < 100) {
501         dirty_background_ratio *= correction;
502         dirty_background_ratio /= 100;
503         vm_dirty_ratio *= correction;
504         vm_dirty_ratio /= 100;
505     }
506     mod_timer(&wb_timer, jiffies + (dirty_writeback_centisecs * HZ) / 100);
507     set_ratelimit();
508 }
```